

❏ 파이썬 1강

파이썬(Python)

1. 웹 개발, 데이터 분석, 인공지능, 자동화 등의 범위에서 활발히 사용되고 있는 언어
2. 인터프리터(interpreter) 형식의 언어 - 작성한 소스 코드를 바로 실행하는 컴퓨터 프로그램 또는 개발 환경

※파이썬은 대표적인 인터프리터 언어, C언어는 대표적인 컴파일 언어

컴파일 (compile): 소스 코드를 기계어로 번역해서 한번에 실행하는 프로그램 또는 개발 환경

3. 데이터의 형태가 실시간으로 적용되는 방식(Dynamic Typing)

※동적(Dynamic) : 런타임 환경에서 변하는 것

▮ ▮ 상황에 따라 추가 제거 가능

정적(Static) : 정해놓은 대로 유지되는 것

▮ ▮ 사용자가 정해놓은 대로 사용

4. 사용이 쉽게 설계되어 있고, 이로 인해 개발 속도가 빠른 편

대부분의 운영체제에서 모두 동작하는 특징이 있다(Window, Linux, Mac)

```
print(10)
print(0.5)
print("a")
print('hello world')
print(10>5)
```

```
10
0.5
a
hello world
True
```

print()는 () 안에 값을 넣으면 출력하는 기능

- 정수, 실수에 대한 입력 가능
- 큰 따옴표, 작은 따옴표로 감싸면 문장 취급
- 식을 출력하는 경우 결과에 따라 True, False 출력

```
print("'오 안녕하시오'")
```

```
print('"반가워요"')
```

```
#은 주석임 a.k.a //
```

```
#은 필요한 상황에만 간단히 남기는 용도
```

```
#컴파일 과정에서 실행 파일 크기에 영향을 주지 않지만 가독성이 떨어짐
```

```
'오 안녕하시오'
"반가워요"
```

```
print("MENTOR Wn 반갑습니다")
print("월Wt 화Wt 수Wt 목Wt 금Wt 토Wt 일")
print(1,2,3,4,5)
print('a','b','c','d','c')
```

```
MENTOR
반갑습니다
월   화   수   목   금   토   일
1 2 3 4 5
a b c d c
```

\n : Enter키

\t : tab키

```
print(1,2,sep="/")
print(1,2,3,4,5,sep="승")
```

```
print("010",1234,5678,sep="-")
print('silverstark0','naver.com',sep="@")
```

```
1/2
1승2승3승4승5
010-1234-5678
silverstark0@naver.com
```

sep는 콤마(,)기능에 대한 구분자를 설정하는 기능

사용자가 별도 설정하지 않는다면 공백 처리

```
print("I Love")
print("Hamburger")

print("I LOVE",end=" ")
print("Hamburger")

print("I think so")
```

```
I Love
Hamburger
I LOVE Hamburger
I think so
```

print()의 end=는 출력 작업 이후에 진행할 작업 사용자가 별도 설정하지 않는다면 줄바꿈(Enter)처리

한번 설정 후 다음 문장 작성할 경우에는 설정이 풀리게 됨

변수

정의: 다양한 값으로 변할 수 있는 수나 데이터를 담는 이름 붙은 저장 공간
메모리 공간

변수를 선언 : 변하는 수 → 변수 → 메모리의 주소를 기억 → Data의 위치 : 리터럴

```
변수타입 변수명 = 리터럴
int a = 3
```

타입(Type) 변수 공간의 크기를 결정하는 방식 (파이썬의 경우 제일 큰 크기를 지정: double 타입)

GarbageCollection(GC) 변수값을 바꿀때마다 데이터값을 기억하고 처리

프로그램이 동적으로 할당한 메모리 중 더 이상 사용되지 않는 영역을

자동으로 해제하는 과정

변수 사용 방법

```
변수명 = 값
(변수명을 선언하고, 이 변수명을 해당 값으로 초기화)
```

변수의 규칙

- 이름이 숫자로 시작할 수 없음
- 특수 문자 사용 불가 [예외: _\$(파이썬 기준)]
- 변수 이름에 공백이 있으면 안됨 [보통 _로 이어짐]
- 특정 예약어를 변수의 이름으로 사용 불가

※예약어(Keyword): if, while, for같이 특정 기능을 수행하는 단어(C언어 기준 약 32가지)

```
#파이썬 내부의 키워드 확인 코드
import keyword
```

```
print(keyword.kwlist)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'fir
```

#파이썬에서는 데이터를 클래스(class)형태로 제공
#클래스는 특정 값을 표현하기 위한 틀

```
number = 1
number2 = 1.5
word = "안녕하시오"
result = 10>5
print(type(number))
print(type(number2))
print(type(word))
print(type(result))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

return : 값을 반환, 가져오는 개념

1. type(값)은 해당 값의 데이터를 return
2. int = 정수
3. float = 실수
4. str = 문자(문자열)
5. bool = 논리(true or false)

```
# 파이썬은 여러 변수에 여러 개의 값을 할당 가능
a = b = c = d = "tomato"
#{ }.format(값)은 {}위치에 a를 넣어주는 기능
# 문자와 값이 섞여있는 출력일 경우 자주 사용 됨
print("a의 값은...{}".format(a))
print("b의 값은...{}".format(b))
print("c의 값은...{}".format(c))
print("{}의 값은...d".format(d))
```

```
a의 값은...tomato
b의 값은...tomato
c의 값은...tomato
tomato의 값은...d
```

```
# a,b = 값1, 값2도 가능
a,b="검", '방패'
print(a,b)
# a,b = b,a 는 값을 서로 교환하는 코드 (Swap)
a,b = b,a
print(a,b)
```

```
검 방패
방패 검
```

파이썬 리터럴(Literal) 표현법

리터럴(literal)은 고정된 값을 표현하는 표현식

#1. 정수(Decimal)

A = 123

#2. 실수(Float)

B = 1.23

#3. 복소수(Complex)

C = 1+23j

#4. 2진수(Binary)

D = 0b11

#5. 8진수(Octal)

```
E = 0o11
```

#6. 16진수(Hexadecimal)

```
F = 0x11
```

#7. 지수(exponent)

```
G = 123e4
```

```
print('정수 ',A)
print('실수 ',B)
print('복소수 ',C)
print('2진수 ',D)
print('8진수 ',E)
print('16진수 ',F)
print('지수 ',G)
```

```
정수 123
실수 1.23
복소수 (1+23j)
2진수 3
8진수 9
16진수 17
지수 123000.0
```

복소수: 실수부 + 허수부로 표현되며 허수부 앞에 j나 J를 붙여줌

(실수를 확장해서 표현하는 경우 사용)

ex)3차 방적식이나 다항 방적식 풀이

진법: 일반적으로 10진법, 다른 진법들은 다음과 같이 적용

2진법 : 컴퓨터의 계산 방식 **0b**

8진법 : 파이썬에서는 **0o**,

16진법 : 기본 10진법 수 뒤에 A~F도 추가 표현 **0x** 1A는 1은 16, A는 10, 값은 26

지수 : aeb로 표현하며 123e3의 경우 123*10의 3제곱으로 표현 1.23은 123e-2로 표현할 수있다

▽ 입력 기능

```
변수명 = input()
```

입력한 값이 문장 형태로 들어감

```
변수명 = input("문장")
```

입력 전에 해당 문장이 창에 뜨게 됨

안내문의 역할

```
변수명 = input("문장을
```

```
여러줄로 작업하는 방법")
```

```
name = input()
print(name)
```

우왕
우왕

```
select_menu = input("주문하시겠습니까? ")
print(select_menu+"를 선택하셨습니다")
```

주문하시겠습니까? 연어덮밥
연어덮밥을 선택하셨습니다!

```
weather_today =input("오늘의 날씨는
대체로..."")
print(f"전국에 {weather_today} 일 것으로 예상됩니다")
#f-String f"{변수명}"으로 적으면 값이 적용
```

```
오늘의 날씨는
대체로...비
전국에 비 일 것으로 예상됩니다
```

```
id = input("아이디를 입력해주세요 >> ")
password = input("비밀번호를 입력해주세요 >> ")
print(f"{id}님 로그인에 성공하셨습니다!")
```

```
아이디를 입력해주세요 >> 연어장덮밥
비밀번호를 입력해주세요 >> ㄹㄷㅏㅈㅅㅈㅈ
연어장덮밥님 로그인에 성공하셨습니다!
```

▽ 파이썬 2강

목차

1. 연산자
2. if - elif - else문
3. while문
4. for문

```
#연산자(operator)
#프로그램 코드에서 특정 연산을 위해 사용하는 기호, 영문자
```

```
#피연산자(operand)
#연산 기호에 의해 연산을 할 대상
```

```
#항
#각각을 이루는 단항식
```

```
a=10
b=8
print(a+b)
#연산자 +
#피연산자 a,b
#항의 개수 2개 (+작업을 위해 필요한 값 a와 b)
```

18

산술 연산자(Arithmetic)

연산자 중에서 수학적 연산을 진행할 때 사용되는 기호

1. + 덧셈
2. - 뺄셈
3. * 곱셈
4. / 나누기 (결과는 실수로 계산)
5. % 나머지 (값을 나누고 남은 값)
6. ** 거듭제곱 ex) a**b
7. // 몫 (정수로 계산) ex) 10//3 = 3

#나머지 기호가 사용되는 예시

```
#사용자가 물건의 금액을 입력하면, 물건을 구매하고 거스름돈을 주는 프로그램 구현
#조건1) 거스름돈은 10000원, 5000원, 1000원으로만 계산
#조건2) 사용자가 가진 금액은 50000원
#출력 예시)
#물건의 금액을 입력해주세요 >> 13000원
#구매할 개수 >> 1개
```

```
#물건을 구매했습니다. 잔액은 37000원입니다.
#거스름돈 처리
#10000원: 3 5000원 : 1 1000원: 2

money = 50000

#int(input()) : 입력한 값(문장)을 정수 형태로 바꾸는 코드
#이 기능을 사용할 경우 정수만 가능
user_money = int(input('물건의 금액을 입력해주세요 >>'))

money = money - user_money

print(f"물건을 구매했습니다.\n남은 금액은{money}입니다.")

# 거스름돈에 대한 계산
man = money // 10000 #10000원짜리 개수 계산
money = money % 10000 #현재 금액을 10000원으로 나누고 남은 값 처리(거스름 돈)
five = money // 5000
money = money % 5000
one = money // 1000
money = money % 1000

print(f"거스름돈 처리\n- 10000원 {man}장 5000원 {five}장 1000원{one}장 - ")
```

```
물건의 금액을 입력해주세요 >>13000
물건을 구매했습니다.
남은 금액은37000입니다.
거스름돈 처리
- 10000원 3장 5000원 1장 1000원2장 -
```

비교 연산자

두 피연산자 간의 관계를 평가하고 비교해
그 결과가 참인지 거짓인지 판단(크기에 대한 비교)

해당 연산은 피연산자가 2개 요구됨 (2항 연산자)

기준은 왼쪽 피연산자에 대한 비교

1. == 같다
2. != 다르다
3. > 초과
4. < 미만
5. >= 이상
6. <= 이하

```
a=5
b=3
print(a==b)
print(a!=b)
print(a>b)
print(a<b)
print(a>=b)
print(a<=b)
```

```
False
True
True
False
True
False
```

대입 연산자(Assignment)

값에 대한 대입에 사용되는 연산자 변수에 값을 할당하거나 기존 값을 변경하는 용도로 사용

1번은 대입 연산자, 나머지는 복합 대입 연산자

$a+=b$ 는 a 에 b 를 더한 결과를 a 에 대입

1. =
2. +=
3. -=
4. *=
5. /=
6. //=
7. %=
8. **=

```
inventory={
    "apple":50,
    "banana":30,
    "carrot":40
}
inventory["apple"] += 10
#shift + alt + 방향키 = 행 복사
inventory["banana"] -= 10
print(inventory)
```

```
{'apple': 60, 'banana': 20, 'carrot': 40}
```

1. 변수명 = {"키1": 값1, "키2": 값2,...}와 같은 데이터를 **dict**라고 한다
2. 변수명[키이름]을 통해 키가 가지고 있는 값을 접근 ex)inventory ← **dict**의 이름
inventory["apple"]은 "apple"을 통해 접근할 수 있는 실제 값(50)
3. print를 통해 **dict**를 출력할 경우,
현재 **dict**에 저장되어있는 내용이 {"키1":값1,"키2":값2}로 출력됨

✓ 논리 연산자(Logic)

프로그래밍에서 주어진 조건이나 표현식을 평가하며,
그 결과로 True 또는 False를 반환(Return)하는 연산자
조건에 대한 결합, 논리적인 흐름에 대한 제어 담당

1. A AND B (둘 다 true인 경우)
2. A OR B
3. NOT A (식의 결과를 반대로 처리)

#논리 연산자 사용 예시

```
cm = 127
age = 12
```

```
#if 조건식 :
#실행할 명령문
#해당 조건식이 True인 경우 실행하는 코드
```

```
if cm < 140 and age < 13 :
    print("해당 놀이 기구는 탑승할 수 없습니다")
```

```
isVIP = False
if not isVIP :
    print("VIP회원만 가능합니다")
```

```
menu = "짜장면"
if menu == "짜장면" or menu == "짬뽕" :
    print("메뉴를 주문했습니다")
```

해당 놀이 기구는 탑승할 수 없습니다
VIP회원만 가능합니다
메뉴를 주문했습니다

#is 연산자
a == b 처럼 a is b로 사용됨

```
a = 0
b = 0
print(a == b)
print(a is b)
```

#차이) ==는 값에 대한 비교, is는 저장된 위치(주소)에 대한 비교
#주소(Address) : 컴퓨터의 저장 공간(memory)에 실제 데이터가 저장되고 있는 영역
#파이썬에서는 값을 할당할 때마다 새 주소를 받음

```
print(hex(id(a)))
#hex : 메모리에서는 주로 16진수로 데이터 표기
#id(값) : 해당 값의 주소 확인
#간혹 값은 같은데 저장위치가 다른 경우도 있음
```

```
print(hex(id(b)))
a = 4096
print(hex(id(a)))
print(a is b)
b = 4095
b += 1 #연산자에 의해서 a와 b의 값이 같아짐
```

```
print(hex(id(b)))
print(a == b)
print(a is b) #is로 계산하면 상황에 따라 True일 수 있고 False일 수 있음
# ==로 계산하면 값을 기준으로 계산하면 동일하게 나옴
```

```
True
True
0xa42648
0xa42648
0x7fdc4bb8afd0
False
0x7fdc4bb8b850
True
False
```

비트 연산자

컴퓨터는 2진수를 기반으로 계산
따라서 같은 계산이어도 2진수의 형태로 연산이 이루어진다면
컴퓨터와 사실상 동일한 환경에서 연산을 진행하며
이는 가장 빠른 연산 속도를 보장한다. True값 대신 1로 도출

1. a & b 비트 AND 연산
2. a | b 비트 OR 연산
3. a ^ b 비트 XOR 연산 (비트가 겹치면 0, 아니면 1로 비트 연산)
4. ~a 비트 NOT 연산 (비트 뒤집기)
5. a << b Left SHIFT 연산 (a의 비트를 b번 왼쪽 이동)
6. a >> b Right SHIFT 연산 (a의 비트를 b번 오른쪽 이동)

#비트 연산 연습
#비트 연산의 경우 실습이 이해하기 쉬움

#2진수 계산 방식
#보통 비트 8개 기준으로 연산 표현
0 0 0 0 0 0 0 0 (값(10진수))

#비트는 맨 오른쪽 값을 기준으로 2의 제곱수로 표현

#1. 2진수로 표현 bin(값)

a = 3

print(bin(a)) #0b11

b = 156

print(bin(b)) #0b1111111

C = 0b11011

print(C) #2진수를 넣으면 자동으로 10진수로 나옴

print(13&9)

#13 = 8+4+1

#9 = 8+0+1

#13&9는 비트를 나열했을 때, 같은 위치에 값이 있으면 그 값을 넣는다

#겹치는 값 8+1= 9

#2진수로 계산법 보기

#0000 1101 (13)

#0000 1001 (9) &

#0000 1001

print(62&17)

print(32|14) #OR 연산 (값이 존재한다면 그 값을 작성)

#0010 0010

#0000 1110

#0010 1110

print(0b00101110)

XOR 연산 (Exclusive OR : 배타적 논리합)

주로 암호화(대칭 키), 오류 검출, 데이터 비교등에서 사용

#장점) 복호화 과정이 간단함

#같은 연산을 한번 더 할 경우 원래의 데이터로 돌아오는 특징이 있음

#입력값과 키가 무작위인 전제라면, 출력값은 입력값과의 연관성이 거의 사라져

#예측이 어렵고 통계적인 공격에 강한 패턴성이 있음(단 키는 절대 재사용 금지)

plain = 123 #평문

key = 99 #키

chiper = plain ^ key #암호화

print("chiper=",chiper)

decoded = chiper ^ key #복호화

print("plain=",decoded)

print(34^14) #XOR 연산 (두 비트가 다르면 1, 같으면 0)

#0010 0010

#0000 1110

#0010 1100

print(0b00101100)

0b11

0b10011100

27

9

16

46

46

chiper= 24

plain 123

#shift 연산
 #왼쪽, 혹은 오른쪽으로 비트를 작성한 칸만큼 이동시킴
 #이동 과정에서 값이 없는 위치는 0을 추가

```
print(0b0011)
print(0b0011<<2)#12
#0011
#0110
#1100
print(0b1100)
print(0b101<<2)
print(0b10100)
print(0b101>>2)
```

```
print(1<<3)
#왼쪽 연산은 2의 지수 연산
print(4>>1)
#오른쪽 연산은 2의 거듭나눔
#결과는 정수
```

```
#x << n == x*(2**n)
#x >> n == x/(2**n)
```

```
#음수에 대한 계산
#파이썬에서는 부호가 있는 2의 보수 방식을 사용
print(-8>>1)
print(-9>>1)
print(-9>>2)
```

```
#2의 보수
#정수를 2진수로 표현할 때, 음수를 나타내는 표준적인 방법
#1. 5 = 0b00000101
#2. 해당 값을 반전
#3. 완성된 값에 +1 진행
#4. 1111 1011
print(bin(-5))
```

```
3
12
12
20
20
1
8
2
-4
-5
-3
-0b101
```

파이썬의 제어문

if문

if 조건문 :

조건이 만족할 때 실행할 명령문

elif 조건문2 :

위의 조건이 맞지 않는 상황에서

조건2가 만족했을 때 실행할 명령문

else :

위의 조건들이 맞지 않는 상황에서 실행할 명령문

파이썬에서는 C언어, JAVA처럼 블록{}으로 작성범위를 표시하지 않음
대신 들여쓰기를 통해 작성 범위를 설정

```
ex)
if a > 10 :
    print("a는 10보다 크다")
else
    print('a는 10보나 작거나 같다')
```

이 경우는 작성 구간이 다르기 때문에 오류 발생

파이썬 3강

자료 구조

데이터를 표현, 관리, 처리하기 위한 묶음형 데이터
데이터 값의 모임, 데이터 값의 관계, 데이터에 적용할 수 있는 특정 명령들

자료 구조의 장점

1. 대규모 데이터에 대한 관리 및 활용
2. 사용자가 요구하는 프로그램에 맞는 맞춤형 알고리즘 설계에 도움을 줌
3. 데이터의 처리 과정을 단순화시키면서 처리 속도도 향상
4. 데이터베이스 등에서 원하는 데이터에 대한 탐색에 용이

시간 복잡도 개념

컴퓨터 과학에서 컴퓨터 프로그램의 입력 값과
연산 수행 시간의 상관 관계를 표현하는 척도
(BIG-O를 통해 표현)

표현법	표현 형태	예제
$O(1)$	상수(constant)	스택(push, pop)
$O(\log_2 n)$	로그(logarithmic)	이진 트리
$O(n)$	선형(linear)	for문
$O(n \log n)$	선형 로그(linear-logarithmic)	퀵, 병합, 힙 정렬
$O(n^3)$	다차(polynomial)	이중 for문, 삽입, 거품, 선택 정렬
$O(3^n)$	지수(exponential)	피보나치 수열
$O(n!)$	팩토리얼(factorial)	

시간 복잡도 성능

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(2^n)$

상수 < 로그 < 선형 < 다항 < 지수

자주 사용되는 자료 구조의 유형

1. 배열(Array)

가장 기본적인 데이터 구조
생성 시 설정된 셀(열)의 수가 고정
각 열에는 인덱스 번호가 부여됨
인덱스를 통해 열 안에 있는 데이터에 접근

ex) 배열 이름 Array, 배열 길이 5
배열 내부의 값 = 64, 58, 77, 95, 80
배열의 인덱스 번호 = 0 ~ 4까지 제공

배열의 장점

1. 만들기 가장 쉽고, 높은 활용도

2. 정렬에 용이함
배열의 단점
3. 데이터의 저장 크기가 고정되어 있음
4. 데이터의 추가/삭제방법이 매우 비효율적

ex) 수학적 벡터(vector), 행렬(matrix) 구현, 스프레드 시트 등의 테이블

2. 스택(Stack)

순서가 보존되는 선형 데이터 구조
가장 마지막에 데이터가 가장 먼저 처리되는 방식으로 구현
(Last in First Out) - LIFO

스택 장점

1. 받은 순서대로 정렬
2. 데이터 크기가 동적
3. 빠른 실행

스택 단점

1. 한번에 하나의 데이터만 처리 가능
2. 가장 최신의 값만 가져오는게 가능

ex) 브라우저 뒤로가기, 실행 취소

큐(Queue)

연결 리스트(Linked List)

해시 테이블(Hash Table)

그래프(Graph)

트리(Tree)

▽ 파이썬에서 제공되는 자료 구조

컬렉션 자료형(Collection)

기본 데이터들을 묶어서 처리하고
관리하기 위한 용도로 제공되는 자료형

```
list1 = [1,2,3,4,5]
#순서 개념 있음 / 인덱스 개념 있음 / 수정 가능
#보편적인 자료 구조

tuple1 = (1,2,3,4,5)
#순서 개념 있음 / 인덱스 개념 있음 / 수정 불가
#일반 배열과 같은 느낌

dict1 = {'name':'Javis','age':23,'Grade':'A'}
#순서 개념 없음 / "A" : B 의 형태로 저장, "A"가 인덱스의 기능 / 수정 가능 / 중복 값 불가
#dict1["name"] = "Javis"
#키와 값이 요구되는 데이터에 사용

set1 = {1,2,3}
#순서 개념 없음 / 기존 dict의 "A":B 구조에서 "A"만 가진 형태 / 수정 가능 / 중복 값 불가
#수학적인 집합 연산에 사용

print(list1)
print(tuple1)
print(dict1)
print(set1)
```

```
[1, 2, 3, 4, 5]
(1, 2, 3, 4, 5)
{'name': 'Javis', 'age': 23, 'Grade': 'A'}
{1, 2, 3}
```

#리스트(list) 사용 방법

```

#1. 리스트 만들기
#리스트명 = [요소1, 요소2....]

list1 = [1,2,3,4,5]
list2 = [] #값을 넣지 않으면 비어있는 리스트 생성
list3 = list() #list()를 통해 리스트 생성 가능
            #리스트가 아닌 데이터를 리스트로 바꿔주는 용도로 더 많이 사용

#2. 리스트 형태로 변환 (list3에 대해)

dict1= {'name':"Javis",'age':23}
list4=list(dict1)
print(list4) # 이 경우 key가 가진 값은 사라지고 key만 등록

#키와 값을 각각 계산해서 저장하는 식으로 처리

keys=list(dict1.keys())
values=list(dict1.values())
print(keys)
print(values)

#형태의 변환은 데이터의 손실이 발생할 수 있는 행위이기 때문에
#필요한 값을 정확히 접근해서 변환하는 것이 중요함

#3. 중첩 리스트(nested list)

nested_list = [[1,2,3],[4,5,6],[7,8,9]]
print(nested_list[0]) #인덱스를 이용한 접근 가능
print(nested_list[0][1])

print(nested_list[1])
print(nested_list[1][2])

nnested_list = [[[1,2,3],[4,5,6],[7,8,9]]]
print(nnested_list[0][2][0])
#nnested_list[0] = [[1,2,3],[4,5,6],[7,8,9]]
#nnested_list[0][2] = [7,8,9]
#nnested_list[0][2][0] = 7

#리스트 슬라이싱(Slicing) - 인덱스 기능 확장
#리스트의 데이터 중에서 필요한 구간만 잘라 저장이 가능
#불필요한 데이터 처리나 특정 데이터만 골라내는데 응용

week = ["Mon","Tue","Wed","Thu","Fri","Sat","Sun"]

#슬라이싱 방법
#리스트명[인덱스 번호 : 마지막 번호+1 : 간격]

#월~수 휴일

holiday=week[0:3]
print(holiday)

#월 수 금 타는 쓰레기

trash=week[0:5:2]
print(trash)

#토~일 주말

# weekend=week[5:]
weekend=week[5:7]
print(weekend)

#Hello
word="Hello World! Today is Sunday"

```

```

w1=word[:5] #인덱스 값을 넣지 않으면 자동으로 0 처리
print(w1)

#Sunday

w2=word[-6:] #값 개수를 적지 않으면 자동으로 마지막 위치까지 범위 설정
              #인덱스 음수 번호는 -1이 전체값의 마지막 위치
              #음수 번호는 마지막 위치에 가까울 경우 음수가 쉬움
print(w2)

#슬라이싱 문법 정리
#a[::]라고 할 경우
#시작 인덱스 : 0
#작업 개수 : 마지막 값 까지
#간격 1

print(word[6:])

```

```

['name', 'age']
['name', 'age']
['Javis', 23]
[1, 2, 3]
2
[4, 5, 6]
6
7
['Mon', 'Tue', 'Wed']
['Mon', 'Wed', 'Fri']
['Sat', 'Sun']
Hello
Sunday
World! Today is Sunday

```

#리스트 연산

```

fruits = ["사과", "딸기", "포도"]
fruits2 = ["파인애플", "체리", "망고"]

#리스트를 더할 경우 순차적으로 합쳐진 '새' 리스트가 생성
basket = fruits + fruits2
print(basket)

basket2 = fruits2 + fruits
print(basket2)

```

```

['사과', '딸기', '포도', '파인애플', '체리', '망고']
['파인애플', '체리', '망고', '사과', '딸기', '포도']

```

```

basket = basket2 - fruits2
#리스트에 대한 뺄셈은 진행 불가

```

```

numbers = [1,2,3,4,5]
numbers = numbers * 3
print(numbers)
#곱하기도 횟수만큼 더하는 뜻이므로 가능

```

```

[1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5]

```

```

numbers[1,2,3,4,5]
numbers = numbers**2
print(numbers)
#제곱은 불가

```

```

numbers=[1,2,3,4]
numbers = numbers/2
print(numbers)
#나눗셈도 불가

```

#리스트 문법

#파이썬에서 리스트 사용을 편하게 하기 위한 문법들을 제공

#1) len(리스트명) : 해당 리스트의 길이(개수)를 확인

```
hands = []  
if len(hands) == 0 :  
    print("현재의 패는 0장...!")
```

#2) 리스트명.append(값) : 해당 값을 리스트에 추가, 이때 추가되는 값은 항상 마지막 데이터

```
hands.append("파멸의 핑킹가위")  
print("카드를 뽑았다!")  
print("현재의 패는", len(hands), "장뿐!")  
print(hands)
```

```
print("마법카드 발동! 첫번째 패에 '은색 색종이'를 추가한다!")
```

#3) 리스트명.insert(인덱스 번호, 값) : 해당 값을 인덱스 번호 위치에 삽입

```
hands.insert(0, "은색 색종이")  
print("현재의 패는", len(hands), "장뿐!")  
print(hands)
```

#4) 리스트명.remove(값) : 해당 값이 리스트에 존재하면 삭제

```
print("카드를 사용했다! 'A', 'B', 'C'를 획득했다")  
hands.remove("파멸의 핑킹가위")  
hands.append("A")  
hands.append("B")  
hands.append("C")  
print("현재의 패는", len(hands), "장뿐!")  
print(hands)
```

#5) 리스트명[인덱스값] = 값 을 통해 해당 위치에 값을 수정

```
print("은색 색종이 사용! 'A'를 'F'로 변경!")  
hands.remove("은색 색종이")  
hands[0] = 'F'  
print("현재의 패는", len(hands), "장뿐!")  
print(hands)
```

#6) 리스트명.pop() : 현재 리스트의 가장 마지막 값을 제거

```
print("상대의 함정카드, 고블린 무녀가 발동! 가장 마지막 위치에 있는 패가 1장 제거!")  
hands.pop()  
print("현재의 패는", len(hands), "장뿐!")  
print(hands)
```

```
print("축복 스테이지가 진행되었습니다. 패가 1,2,3,4,5로 변경됩니다.")
```

```
hands = [1,2,3,4,5]  
print(hands)
```

#7) 리스트명.clear() : 현재 리스트 내부에 있는 모든 값이 삭제

```
print("상대의 카드 광전사의 영혼 발동! 패를 전부 패버립니다")  
hands.clear()  
print("현재의 패는", len(hands), "장뿐!")  
print(hands)
```

del hands #리스트 삭제

```
print("패배!")
```

```
현재의 패는 0장...!  
카드를 뽑았다!  
현재의 패는 1 장뿐!  
['파멸의 핑킹가위']  
마법카드 발동! 첫번째 패에 '은색 색종이'를 추가한다!  
현재의 패는 2 장뿐!  
['은색 색종이', '파멸의 핑킹가위']  
카드를 사용했다! 'A', 'B', 'C'를 획득했다  
현재의 패는 4 장뿐!  
['은색 색종이', 'A', 'B', 'C']  
은색 색종이 사용! 'A'를 'F'로 변경!  
현재의 패는 3 장뿐!
```

```
['F', 'B', 'C']
상대의 함정카드, 고블린 무녀가 발동! 가장 마지막 위치에 있는 패가 1장 제거!
현재의 패는 2 장뿐!
['F', 'B']
축복 스테이지가 진행되었습니다. 패가 1,2,3,4,5로 변경됩니다.
[1, 2, 3, 4, 5]
상대의 카드 광전사의 영혼 발동! 패를 전부 패버립니다
현재의 패는 0 장뿐!
[]
패배!
```

#연습문제 1) 1부터 10까지 순서대로 값을 가지고 있는 리스트 numbers에서 홀수만 출력할 것

```
numbers = [1,2,3,4,5,6,7,8,9,10]
```

```
solo = numbers[::2]
print(solo)
```

```
[1, 3, 5, 7, 9]
```

#연습문제 2) 음식 메뉴 이름에 대한 리스트와 음식 가격에 대한 리스트를 구현하고,
#화면에서 입력을 통해 물건을 구매하는 프로그램을 설계할 것

```
foods = {"짜장면":5000, "마라탕":10000, "짬뽕":6000, "탕수육":9000}
```

```
MENU = input("메뉴를 골라주세요 :")
price = foods[MENU]
if MENU in foods:
    print(f"{MENU}: {price}원")
    print(f"{MENU}를 주문하시겠습니까?")
    answer = input("Y/N : ")
    if answer == "Y" or answer == "y" :
        print("주문이 완료되었습니다.")
    if answer == "N" or answer == "n":
        print("주문이 취소되었습니다.")
    else:
        print("잘못된 입력입니다.")
else:
    print("없는 메뉴입니다.")
```

```
메뉴를 골라주세요 :짜장면
짜장면: 5000원
짜장면을 주문하시겠습니까?
Y/N : n
주문이 취소되었습니다.
```

#연습문제 3) 1부터 100사이의 값 중 3과 5의 공배수로만 이루어진 리스트 A를 생성할 것

```
a = []
for i in range(1,101):
    if i%3 == 0 and i%5 ==0:
        a.append(i)
print(a)
```

```
[15, 30, 45, 60, 75, 90]
```

```
a = 11
if a > 10:
    print("a는 10보다 크다")
else : #옳은 예
    print("a는 10보다 작거나 같다")
```

```
a는 10보다 크다
```

if문 사용 시 유의사항

- if문 다음에 if문을 작성할 경우 각각 독립된 문법으로 처리
- if - elif - else를 반드시 작성할 필요는 없음

- elif 또는 else는 if없이 단독 사용 불가
- 작성 범위(들여쓰기)에 유의

#각각 독립된 문법으로 처리되는 경우

```
times = 670
if times > 600 :
    print("time over")
if times >= 600 :
    print("time over too")
```

#결론 : if문은 각각 설계됨

```
times = 500
if times > 500 :
    print("time 500")
elif times >= 500 :
    print("time 500 too")
```

#결론 : 연결된 하나의 조건의 명령문을 만들 경우 if - elif로 설계 진행

```
time over
time over too
time 500 too
```

#연습 1. 다음 조건에 맞는 프로그램 코드를 완성할 것

- #조건 1. number에 정수 입력
- #조건 2. number에 양수인지 음수인지 출력
- #조건 3. number가 0인 경우 0으로 출력

```
number = int(input('숫자를 입력해주세요 >> '))

print(number)
if number > 0:
    print("양수")
elif number < 0:
    print("음수")
else :
    print("0")
```

```
숫자를 입력해주세요 >> 45
45
양수
```

#날짜 / 시간 관련 기능

import datetime as d #datetime을 d로 사용

```
now = d.datetime.now()

print(now.year,"년")
print(now.month,"월")
print(now.day,"일")
print(now.hour,"시")
print(now.minute,"분")
print(now.second,"초")
#UTC 기준, 한국 표준과 9시간 정도 차이

print(f"현재 시간은 {now.hour+9}시{now.minute}분입니다")
```

```
2025 년
7 월
12 일
1 시
37 분
5 초
현재 시간은 10시37분입니다
```

#현재 날짜와 시간을 기준으로 오전과 오후를 구분하는 프로그램 작성
#조건) 24시가 아닌 12시 기준으로 출력할 것

#ex) 14시 → 2시

출력 예시)

현재 시간은 오전 10시 38분입니다

```
import datetime as today
now = today.datetime.now()

if now.hour >= 12:
    print(f"오후 {now.hour-12}시 {now.minute}분")
else:
    print(f"오후 {now.hour}시 {now.minute}분")

#과제) 복습때 현재 작성된 코드에서의 문제점이 발견된다면
#수정코드는 아래에 따로 작성
```

오후 1시 41분

While문

특정 조건이 만족하는 동안

계속적으로 반복적으로 명령을 수행하는 제어문

```
while 조건 :
    실행할 명령문
    (true일 동안 작업)
```

```
i=1
while i<=5:
    print(i)
    i+=1 #while문을 적을 때는 의도가 아니라면 조건을 변경할 코드가 작성되어 있어야 함
```

1
2
3
4
5

while문 작성 시 유의사항

- 무한 루프 주의 (조건이 true가 지속되면 멈추지 않으므로)
- 조건에 대한 작성을 위해 변수를 만들어줄 것
- 특정 시점을 멈추기 위한 break 코드 작성
- C/JAVA에서 제공되는 do-while이 파이썬에서는 없음

do-while이란.. C/JAVA에서 사용하는 방식

```
do
{
    무조건 1번은 실행할 코드
    while(조건):
}
```

조건을 나중에 검사하는 방식으로 무조건적인 1회 실행을 보장

```
#break, continue → 반복적인 코드에서 사용하는 제어문
#break: 작업종료
#continue: 현재 반복 생략 후 다음 반복으로

#while True는 무한 반복 코드
while True :
    a=input("멈추고 싶다면 종료를 입력")
    if a == "종료":
        break
```

#while문과 들여쓰기가 다르기 때문에 이 코드는 while문이 마무리가 되었을 때 실행
print("while문 종료")

멈추고 싶다면 종료를 입력종료

while문 종료

1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51 53 55 57 59 61 63 65 67 69 71 73 75 77 79 81 83 85 87 89 91 93 95 97
프로그램 종료

```
i=1
while i <= 100:
    if i%2==0:
        i+=1
        continue
    print(i,end=" ")
    i+=1
print() #print()를 그냥 작성하는 경우 end는 enter, sep는 띄어쓰기로 자동 설정
print("프로그램 종료")
```

1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51 53 55 57 59 61 63 65 67 69 71 73 75 77 79 81 83 85 87 89 91 93 95 97
프로그램 종료

```
i = 1
while i <= 30:
    if "3" in str(i) or "6" in str(i) or "9" in str(i):
        print("짝",end=" ")
    else:
        print(i,end=" ")
    i+=1
print()
print("프로그램 종료")
```

1 2 짝 4 5 짝 7 8 짝 10 11 12 짝 14 15 짝 17 18 짝 20 21 22 짝 24 25 짝 27 28 짝 짝
프로그램 종료

#연습 문제) 반복문을 이용하여 1~30까지의 숫자를 반복합니다
3,6,9가 숫자로 나오는 경우 숫자대신 "짝" 입력
출력은 한 줄로 처리

```
i=1
while i <= 30:
    tens = i//10 #10의 자리
    units = i%10 #1의 자리

    #1의 자리가 3,6,9 / 10의 자리가 3,6,9인 경우
    if tens ==3 or tens==6 or tens==9 or units==3 or units==6 or units==9:
        #if ones in [3,6,9] or tens in [3,6,9] :
        print("짝",end=" ")
        #tens == 3 or 6 or 9와 같은 방식으로는 작성 불가

    else:
        print(i,end=" ")

    i+=1

print()
print("끝")
```

1 2 짝 4 5 짝 7 8 짝 10 11 12 짝 14 15 짝 17 18 짝 20 21 22 짝 24 25 짝 27 28 짝 짝
끝

▼ for문

시퀀스를 순회할 때 사용하는 반복문

※파이썬의 시퀀스는(Sequence)는

리스트, 문자열, range 함수 등 순서가 있는 묶음형 데이터

ex) list(), tuple(), range(),str()

for 변수 in 시퀀스 :
실행할 코드

변수에는 시퀀스의 각각의 요소가 반복해서 하나씩 처리되는 방식

C/JAVA의 for문

```
for(int i = 0; i<=30; i++)  
{  
    //실행할 문법  
}
```

#1. range(start,end)를 활용한 for문
#start부터 "end -1" 까지 숫자형 묶음 데이터
print(type(range(1,6))) #range 형태의 데이터
print(range(6,11))

```
for i in range(1,6):  
    print(i,end = "슴")  
print(" 비비")  
for i in range(6,11):  
    print(i,end="작")  
print(" 따잇")
```

```
<class 'range'>  
range(6, 11)  
1슴2슴3슴4슴5슴 비비  
6작7작8작9작10작 따잇
```

#2. range(end)를 활용한 for문
#start가 없으면 0부터 end -1까지 숫자형 묶음 데이터

#내부에서 따로 i 자체를 사용하지 않는다면, 단순 횟수 반복같은 느낌으로 사용

```
for i in range(5):  
    print("할할할할할")  
for i in range(9):  
    print(i,end=" ")
```

```
할할할할할  
할할할할할  
할할할할할  
할할할할할  
할할할할할  
0 1 2 3 4 5 6 7 8
```

#3. range(start, end, step)을 활용한 for문
#주로 짝수나 홀수처럼 간격 이동을 구현할 때 사용

#start : 시작 end : 마지막 -1 step : 간격
#따로 설정이 없는 경우(start=0,step=1)

```
for i in range(1,11,2):  
    print(i,end=" ")  
print("작업 1 종료")  
for i in range(1,11,3):  
    print(i,end=" ")  
print("작업 2 종료")
```

```
1 3 5 7 9 작업 1 종료  
1 4 7 10 작업 2 종료
```

#4. 문자열

```
for i in "Hot6":
```

```
print(i,end = " ")
print("작업 종료")
```

Hot 6 작업 종료

#5. 리스트 ★

#for문이 가장 많이 사용하는 데이터 유형
#[1,2,3]과 같이 표현

```
food = ["사과","바나나","딸기"]
```

```
for i in food :
    print(i,end=" ")
print("옴뇽뇽")
```

사과 바나나 딸기 옴뇽뇽

#6. 리스트 + enumerate()

#enumerate()를 통해 묶음을 처리하면, 순번(인덱스)과 값을 동시에 뽑을 수 있음

```
fruits = ["사과","바나나","딸기"]
```

```
for idx, fruits in enumerate(fruits, start=1):
    print(idx,fruits)
```

1 사과
2 바나나
3 딸기

#7. 리스트+zip()

#zip()은 시퀀스끼리 묶어주는 역할

```
member = ["비비","란","나이아","빅우드"]
types = [20,21,19,30]
```

```
for m, t in zip(member,types):
    print(f"{m}: 체력{t}")
```

```
print(zip(member,types))
```

```
datas = zip(member,types)
datas = list(datas)
print(datas)
```

비비: 체력20
란: 체력21
나이아: 체력19
빅우드: 체력30
<zip object at 0x7e35159ffa80>
[('비비', 20), ('란', 21), ('나이아', 19), ('빅우드', 30)]

```
KOR = [100,100,95]
```

```
JPN = [80,70,92]
```

```
CHN = [10,150,55]
```

```
for (k,j,c) in zip(KOR,JPN,CHN):
    print(f"{k}.총점:{k+j+c} 평균: {(k+j+c)/3}")
```

```
print()
```

```
for i,(k,j,c) in enumerate(zip(KOR,JPN,CHN)):
    print(f"{i+1}.총점:{k+j+c} 평균: {(k+j+c)/3}")
```

100.총점:190 평균: 63.333333333333336
100.총점:320 평균: 106.66666666666667
95.총점:242 평균: 80.66666666666667

1.총점:190 평균: 63.333333333333336
2.총점:320 평균: 106.66666666666667
3.총점:242 평균: 80.66666666666667

▽ 파이썬 4강

▽ 함수(Function)

프로그램 내에서 반복적으로 사용되는 가치있는 부분을
한 뭉치로 묶어 값을 제공할 때
특정 결과 값이 처리되도록 설계하는 명령문 집합체

ex) 과일(입력) → 믹서기(함수) → 주스(출력)

특정 작동 알고리즘 구현

$a = 2b + 3$ 이라고 할 때,
b가 10이라면 a는 5

수학적인 공식 구현

함수 설계의 목적

- 중복적인 코드 설계 방지
프로그램의 흐름을 파악하기 용이
→ (코드의 가독성, 재사용성 향상)
- 오류가 날 경우, 그 부분만 분리해서 유지보수에 적절
→ (코드의 모듈화)
- 전체적인 설계를 함수로 통해 설계하면
복잡한 작업들을 간단하게 표현할 수 있음
→ (추상화)

파이썬 함수의 구조

```
def 함수_이름(매개변수) :  
    실행할 명령문  
    ...
```

```
def add(a,b) :  
    return a+b
```

def → 함수를 표현하는 예약어
add → 함수의 이름
a,b → 매개변수의 이름
return → 함수를 종료하고 다음에 적힌 값을 함수의 호출 위치에 전달

함수의 이름 : add
매개변수: a,b (2개 값 요구)
출력값: return (a+b)

```
a = add(1,5)  
print(a)  
print(add(2,5))
```

6
7

```
def sample() :  
    print("sample test...")
```

```
sample()
```

```
sample test...
```

함수의 종류

1. 일반 함수(procedure) : return이 없는 함수, 실행하면 바로 특정 기능 수행
2. return 함수(function) : return값이 작성되는 함수, 기능 수행 후 값을 호출한 위치로 전달
3. 재귀 함수(Recursive) : 함수 내에서 자기 자신을 호출하는 형태의 함수
4. 내장 함수(Built-in) : 프로그래밍 언어에서 기본적으로 제공해주는 함수
ex) print()
5. 사용자 정의 함수 : 사용자가 직접 정의해 사용하는 함수

#함수 연습 1

#매개변수 2개를 요구하며, 실행 시 두 값의 곱하기 값을 return하는 함수 multiple을 설계할 것

```
def multiple(a,b):  
    return(f'{a}*{b}={a*b}')
```

```
print(multiple(2,6))
```

```
2*6=12
```

함수의 기본 형태

1) 매개변수를 요구하는 함수

해당 함수를 호출할 경우 반드시 매개변수에 해당하는 값(인자)을 넣어서 호출

```
def 함수_이름(매개변수):  
    함수 호출 시 실행할 코드 작성 영역  
    ...  
    return 값
```

2) 매개변수를 요구하지 않는 함수

해당 함수를 호출할 경우 전달해주는 값 없이 바로 사용

```
def 함수_이름():  
    함수 호출 시 실행할 코드 작성 영역  
    ...  
    return 값
```

3) 매개변수를 요구하나 return이 없는 함수

해당 함수에는 return 값이 따로 없음

```
def 함수_이름():  
    함수 호출 시 실행할 코드 작성 영역  
    ...
```

매개변수로 정수를 전달받으면 그 수만큼 폭★8을 출력하는 함수 print_star를 구현하세요

ex) print_star(4) 폭★★★★8

#함수 연습 2

#print로 문법적 처리

```
def print_star(a):  
    print(f"폭{'★'*a}8")
```

```
#return을 이용한 처리(실제 출력(print)은 아님)
def print_star2(a):
    return f"꼭{'★'*a}8"

print_star(4)

print_star2(3)
print(print_star2(4))
```

```
꼭★★★★8
꼭★★★★8
```

다음 조건을 만족하는 함수를 구현하세요

- 조건 1. 함수 이름은 getArea
- 조건 2. 매개변수의 이름은 r
- 조건 3. 이 함수에서 원주율 값(pi)는 3.14
- 조건 4. 매개변수를 넣고 해당 함수를 호출 시 원의 둘레

실행 : getArea(10)

```
def getArea(r):
    return 2*3.14*r
#소수점 2자리까지 제한해서 출력하고 싶은 경우
#return f"{2*3.14*r:.2f}"
```

```
print(getArea(10))
#소수점 N자리까지 제한해서 출력하고 싶은 경우
#f"{값}:.Nf"

print(f"{getArea(10):.2f}")
```

```
62.800000000000004
62.80
```

```
#a+aa+aaa를 계산하는 함수 aby3을 구현하세요
#풀이 1) 수학적 규칙 이용하기
def aby3(a):
    # return a+(a+10*a)+(a+10*a+100*a)
    # return a+a(10+1)+a(100+10+1)
    # return a(1+11+111)

    #2자리 수부터는 적용이 안되니 주의
    #숫자 범위 제한이 있는 경우에만 가능한 방법
    return a*123
```

```
a = int(input('1부터 9사이의 값을 입력해주세요 >> '))
print(aby3(a))
```

```
1부터 9사이의 값을 입력해주세요 >> 10
1230
```

```
#풀이 2) 문자열 이용하기
#파이썬에서 문자열은 "곱할" 경우 이어붙여짐
#정수 1 → 문자 "1"로 변경
#문자열로 계산
#문자 "1" → 정수 1로 변경
#return
```

```
def aby3(a):
    s = str(a) #1 = "1"
    return int(s)+int(s*2)+int(s*3)
```



```
a = int(input('1부터 9사이의 값을 입력해주세요 >> '))
print(aby3(a))
```

```
1부터 9사이의 값을 입력해주세요 >> 1
123
```

#풀이 3) 자릿수를 구하는 규칙으로 풀이

```
def aby3(a):
    #a의 자릿수 구하기
    #정수 a의 자릿수 → 10으로 몇번 나누어지는가
    #ex) 1234 - 123
    #    123 - 12
    #    12 - 1
    #    1 - 0
    #자릿수는 4자리
    digit = 0 #자리수 표현
    temp = a #임시값(교환용)
    while temp>0:
        temp = temp//10 #값을 10으로 나눈 몫
        digit += 1 #자리수 증가
        #aa = a&(10^자리수+1)
        #aaa = a&(10^(자리수*2)+10^자리수+1)
        p = 10**digit #자릿수의 제곱
        return a+a*(p+1)+a*(p**2+p+1)
```

```
a = int(input('1부터 9사이의 값을 입력해주세요 >> '))
print(aby3(a))
```

```
1부터 9사이의 값을 입력해주세요 >> 12
1476
```

함수 파트에서 프로그래밍 코드를 작성할 경우
다양한 방식으로 설계가 가능하므로
처음에는 본인이 구현 가능한 방법으로 이해하고,
다음에는 다른 풀이를 보고 분석하기,
마지막으로 각 코드별 성능 분석하기,
최종적으로 가장 성능이 좋은 코드를 목표로 설계하기

조건1. 20% 농도의 소금물 100g과
물 200g을 혼합한 소금물의 농도(%)를 소수점 2번째 자리까지 계산하세요

조건2. 함수의 이름은 getSalt

힌트1. "f"{salt:.2f}"

힌트2. 소금물의 농도 = 소금의 양 / 소금물의 양*100

```
def getSalt(water):
    salt = 20/(100+water)*100
    print(f"{salt:.2f}%")

getSalt(200)

#소금의 양 = 소금물의 농도 * 소금물의 양/100
# salt = (20 * 100) / 100 = 20 (물 200g은 소금물과 상관 없으므로 계산에서 제외)

#소금물의 농도 = 20 / 300

def getSalt2(salt,water):
    return salt / water * 100

print(f"{getSalt2(20,300):.2f}%")
```

6.67%
6.67%

문제) 2 이상의 양의 정수를 하나 입력 받아서,
해당 숫자가 소수인지 판단하세요

ex)

check(3) → "소수..소수를 세자..."

check(4) → "크읏... 소수가 아니잖아!"

```
def check(A):
    for i in range(2, A):
        if A % i == 0:
            print("크읏... 소수가 아니잖아!")
            return
    print("소수..소수를 세자...")
    return
```

```
check(4)
check(5)
check(1)
```

```
크읏... 소수가 아니잖아!
소수..소수를 세자...
소수..소수를 세자...
```

▽ 파이썬 5강

▽ 튜플(Tuple)

특징

1. 튜플은 가지고 있는 요소를 생성, 추가, 삭제가 불가능
2. 리스트는 [], 튜플은 ()로 작성

사용 목적

1. 데이터 불변성

튜플은 생성된 이후 내부 요소에 대한 변경이 불가능해
실수로 값이 변경되는 상황을 사전에 방지할 수 있음 ex) 좌표, 설정값

2. 메모리 효율성

리스트와 달리, 추가적인 작업을 위한 여분이 필요가 없는 데이터로
리스트보다 메모리를 덜 사용하고, 연산 속도도 더 빠름

3. 여러 개의 값을 동시에 변수에 할당하는 경우 또는 여러 개의 값을 추출하는 경우 튜플이 사용됨

할당 : 패킹
추출 : 언패킹

#튜플 생성

```
t = (1,2,3) #튜플명 = (값1,값2,값3...)
print(t)
```

```
#파이썬에서는 ()괄호가 없으면 기본적으로 튜플로 인식
t1 = 3,4,5
print(t1)
```

```
t2 = () #empty
print(t2)
```

```
t3 = tuple(['a','b','c']) #list → tuple
print(t3)
```

```
print(type(t))
print(type(t1))
print(type(t2))
print(type(t3))
```

```
(1, 2, 3)
(3, 4, 5)
()
('a', 'b', 'c')
<class 'tuple'>
<class 'tuple'>
<class 'tuple'>
<class 'tuple'>
```

#인덱스 / 슬라이싱

```
print(t[0]) #튜플도 리스트와 마찬가지로 순서 개념이 존재 (인덱스 활용 가능)
print(t3[:2]) #인덱스의 확장인 슬라이싱도 가능
```

```
1
('a', 'b')
```

#튜플의 합과 곱

```
t4 = t3*3
print(t4)
```

```
t5 = t1+t3
print(t5)
```

```
#튜플의 길이
print(len(t5))
```

```
('a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c')
(3, 4, 5, 'a', 'b', 'c')
6
```

#값 제거

```
del t5(0) #불가능 (튜플은 생성 시점 이후로 추가,삭제 작업 불가)
```

```
del t5 #튜플 자체를 삭제하는 건 가능
```

[숨겨진 출력 표시](#)

▼ 딕셔너리(Dict)

특징

1. "제품명":"만년필"과 같이 대응 관계로 값을 표현

■ ■ "제품명"은 키(key), "만년필"은 값(value)에 해당

키(key)

데이터에 접근하기 위한 고유한 식별자,
중복 데이터 불가 각 키는 고유한 값을 지님 ex) 상품 코드, ID, 모델...

값(value)

실제로 저장되는 데이터 자체 값, 키와 연결되어 저장되고 키를 통해 접근할 수 있음 (대응관계)

키와 값이 사용되는 경우

1. 딕셔너리 : 키-값을 쌍으로 저장하는 자료 구조
2. 해시 테이블 : 키를 해시 함수를 통해 인덱스로 변환,
값을 저장하는 자료 구조
3. JSON : 데이터 교환 수단의 텍스트 기반 형식
해당 데이터는 키-값을 통해 데이터를 표현

딕셔너리가 사용되는 경우

1. 데이터베이스 연동 작업

데이터베이스 테이블의 레코드를 표현할 때 사용

2. 데이터 분석

키를 통해 값을 바로 접근할 수 있어 매우 빠른 데이터 검색이 가능,
키-값을 통한 데이터 간의 관계 표현이 쉬움

3. 메모이제이션

함수 호출 결과를 딕셔너리에 따로 저장해, 동일한 입력에 대한 재계산을 방지
ex)

- 함수 호출
- 함수에서 조사한 값을 딕셔너리에 저장
- 함수 재 호출 시 기존 딕셔너리를 통해 조사하고
- 그 값이 존재할 경우 계산에서 배제

4. 단어의 빈도 수 계산

단어를 키로 빈도 수를 값으로 저장,
각 단어의 빈도 수를 체크하는데 효과적

#딕셔너리 만들기

#딕셔너리명 = {키1: 값1, 키2:값2....}

```
my_dict = {"name":"max Codian","phone_number":"010-0101-0101","regist":"2025-07-20"}
```

#딕셔너리를 활용해 표 출력 - 판다스(pandas) 연결

#판다스는 데이터 분석과 처리를 위해 제공되는 파이썬 라이브러리 파일

#주로 시리즈(Series)와 데이터 프레임(DataFrame)이라는 데이터 구조를 사용

#시리즈 = 1차원, 데이터 프레임 = 2차원

#List → Series, Dict → DataFrame

```
import pandas as pd
```

```
#import 라이브러리명 as alias(별칭)
```

```
#>>프로그램 내에서 해당 라이브러리를 별칭으로 대신 사용 선언
```

```
#dict → DataFrame
```

```
data_frame = pd.DataFrame([my_dict]) #데이터 프레임은 행과 열로 구성된 2차원 표
```

#표 형태로 출력

```
print(data_frame.to_string()) #일반 표
```

```
print()
```

```
print(data_frame.to_string(index=False)) #인덱스 제외
```

```
print()
```

```
print(data_frame.to_html()) #HTML 테이블 형태
```

```
print()
print(data_frame.to_markdown()) #마크다운
```

```
      name  phone_number  regist
0 max Codian  010-0101-0101  2025-07-20
```

```
      name  phone_number  regist
max Codian  010-0101-0101  2025-07-20
```

```
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>name</th>
      <th>phone_number</th>
      <th>regist</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>max Codian</td>
      <td>010-0101-0101</td>
      <td>2025-07-20</td>
    </tr>
  </tbody>
</table>
```

	name	phone_number	regist
0	max Codian	010-0101-0101	2025-07-20

#데이터 프레임(행 X 열 확인)

```
print(data_frame.shape) #(행, 열)
print(data_frame.columns) #열 이름 확인
print(data_frame.index) #인덱스
```

```
(1, 3)
Index(['name', 'phone_number', 'regist'], dtype='object')
RangeIndex(start=0, stop=1, step=1)
```

#열 데이터 선택

```
data1 = data_frame['name']
print(data1)

datas = data_frame[['name', 'phone_number']]
print(datas.to_markdown())
```

```
0    max Codian
Name: name, dtype: object
|  | name      | phone_number |
|---|:-----|:-----|
| 0 | max Codian | 010-0101-0101 |
```

#딕셔너리 관련 문법

```
my_dict = {"name": "jake"}
#값 추가 기능 : 딕셔너리명[키] = 값
my_dict["ID"] = "Heywood"
```

```
print("표1")
df = pd.DataFrame([my_dict])
print(df.to_markdown())
```

#값 추가 과정에서 '키'가 이미 존재한다면,
#키는 중복될 수 없으므로 값만 수정됨

```
print("표2")
my_dict["name"] = "melto"
df = pd.DataFrame([my_dict])
print(df.to_markdown())
```

```
#키-값 제거 기능 del 딕셔너리명[키]
del my_dict["ID"]
```

```

print("표3")
df = pd.DataFrame([my_dict])
print(df.to_markdown())

#내용 제거

my_dict.clear()

if not my_dict: #if문에서 자료구조를 조건으로 넣으면, 데이터의 여부를 확인
    #not이 붙었다면 데이터에 값이 없다면?(빈 값 체크)로 해석 가능
    print("딕셔너리가 비었음")

#전체 제거

del my_dict

#특정 키가 딕셔너리 안에 있는지를 조사하는 방법(in)

a = {"name" : "Sarah", "phone": "101-0100-1110", "birth": "1108"}

#"job"키가 딕셔너리 안에 없다면, 디폴트 키인 "job"의 값을 출력하세요
#디폴트 키 : 딕셔너리에 포함되지 않는 미리 정해둔 기본 값
#주로 특정 값이 없을 때 대신 임시로 사용할 값에 대한 설정에 사용

if "name" in a:
    print(a["name"])
if not 'job' in a:
    print(a.get("job", "They took our Job!!!"))
print(a)

```

```

표1
|  | name | ID      |
|---|:-----|:-----|
| 0 | jake  | Heywood |

표2
|  | name | ID      |
|---|:-----|:-----|
| 0 | melto | Heywood |

표3
|  | name |
|---|:-----|
| 0 | melto |
딕셔너리가 비었음
Sarah
They took our Job!!!
{'name': 'Sarah', 'phone': '101-0100-1110', 'birth': '1108'}

```

```

#딕셔너리를 간단히 만드는 방법
#유의사항 : 키는 "문자열"로 만들어 줄 것
#딕셔너리명 = dict(키1=값1, 키2=값2, 키3=값3...)
sample = dict(a=1,b=2,c=3)
#a를 "a"로 안 써도 되어 쉽게 작성 가능
#sdf = pd.DataFrame([sample])
#print(sdf.to_markdown())

#딕셔너리 컴프리헨션(dictionary Comprehension)
#딕셔너리명 = {변수명:값 for문 if문...}
sample2 = {x:x**2 for x in (1,2,3)}
#x는 1,2,3에 각각 적용
sdf = pd.DataFrame([sample2])
print(sdf.to_markdown())

```

```

|  | 1 | 2 | 3 |
|---|:---|:---|:---|
| 0 | 1 | 4 | 9 |

```

```

sample = {
    "이름":["소라", "한솔", "유영"],
    "나이":[25,20,22],
    "거주지":["인천", "서울", "순천"]
}

```

```
df = pd.DataFrame(sample)
print(df.to_markdown())
```

	이름	나이	거주지
0	소라	25	인천
1	한솔	20	서울
2	유영	22	순천

```
import pandas as pd
```

```
A = {
    "이름": [],
    "성별": [],
    "학년": [],
    "반": [],
    "점수": []
}
for i in range(1,6):
    A["이름"].append(input("이름을 입력해주세요 >>"))
    A["성별"].append(input("성별을 입력해주세요 >>"))
    A["학년"].append(input("학년을 입력해주세요 >>"))
    A["반"].append(input("반을 입력해주세요 >>"))
    A["점수"].append(input("점수를 입력해주세요 >>"))
    i+=1
    print("입력완료!\n")
    NEW = pd.DataFrame(A)
    print(NEW.to_markdown(), "\n")
```

```
이름을 입력해주세요 >>김철수
성별을 입력해주세요 >>남
학년을 입력해주세요 >>1
반을 입력해주세요 >>1
점수를 입력해주세요 >>85
입력완료!
```

	이름	성별	학년	반	점수
0	김철수	남	1	1	85

```
이름을 입력해주세요 >>김철수
성별을 입력해주세요 >>남
학년을 입력해주세요 >>1
반을 입력해주세요 >>2
점수를 입력해주세요 >>85
입력완료!
```

	이름	성별	학년	반	점수
0	김철수	남	1	1	85
1	김철수	남	1	2	85

```
이름을 입력해주세요 >>김철수
성별을 입력해주세요 >>남
학년을 입력해주세요 >>1
반을 입력해주세요 >>1
점수를 입력해주세요 >>85
입력완료!
```

	이름	성별	학년	반	점수
0	김철수	남	1	1	85
1	김철수	남	1	2	85
2	김철수	남	1	1	85

```
이름을 입력해주세요 >>김철수
성별을 입력해주세요 >>여
학년을 입력해주세요 >>1
반을 입력해주세요 >>1
점수를 입력해주세요 >>90
입력완료!
```

	이름	성별	학년	반	점수
0	김철수	남	1	1	85
1	김철수	남	1	2	85
2	김철수	남	1	1	85
3	김철수	여	1	1	90

```
이름을 입력해주세요 >>김철수
```

성별을 입력해주세요 >>?
학년을 입력해주세요 >>1
반을 입력해주세요 >>1
점수를 입력해주세요 >>78
입력완료!

| | 이름 | 성별 | 학년 | 반 | 점수 |

#강사의 풀이

=====

#1) 저장방식이 여러번 반복되므로 '함수'로 설계

#ist : 키 리스트에 대한 저장용

#key : 데이터

#mode: 숫자인지 아닌지 여부

```
def input_data(ist,key,mode):  
    if mode == True:  
        data = int(input(f"{key}을(를) 입력해주세요 >> "))  
    else :  
        data = input(f"{key}을(를) 입력해주세요 >> ")  
    ist.append(data)
```

=====

#2) 데이터 설계

```
name_=[]  
gender_=[]  
grade_=[]  
class_=[]  
score_=[]
```

=====

#3) 작업(5번) : for문

```
for i in range(5):  
    input_data(name_,"이름",False)  
    input_data(gender_,"성별",False)  
    input_data(grade_,"학년",True)  
    input_data(class_,"반",True)  
    input_data(score_,"점수",True)  
    print("입력 완료!")
```

=====

#4) dict → DataFrame(using pandas)

```
sample = {  
    "이름":name_,  
    "성별":gender_,  
    "학년":grade_,  
    "반":class_,  
    "점수":score_  
}
```

```
df = pd.DataFrame(sample)
```

=====

#5) 결과 출력

```
print(df.to_markdown())
```

다음 표를 보고, 주어진 조건대로 작성하세요

이름 가격

월드콘 1500

끌레도르 2400

메로나 1200

조스바 1300

#문제 1) 해당 표를 기준으로 딕셔너리 cone을 구현하세요

```
#일반 풀이
cone = {"월드콘":1500,"글레도르":2400,"메로나":1200,"쥬스바":1300}
print(cone)

#판다스의 경우
import pandas as PD

cone = {
    "이름":["월드콘","글레도르","메로나","쥬스바"],
    "가격":[1500,2400,1200,1300]
}

ice = PD.DataFrame(cone)
print(ice.to_markdown())
```

```
{'월드콘': 1500, '글레도르': 2400, '메로나': 1200, '쥬스바': 1300}
|   | 이름   |   가격   |
|---|:-----|:-----:|
| 0 | 월드콘   |   1500   |
| 1 | 글레도르 |   2400   |
| 2 | 메로나   |   1200   |
| 3 | 쥬스바   |   1300   |
```

#문제 2) 쥬스바의 가격은 1200원입니다. 현재 표에서 1300원이므로 1200원으로 수정해주세요
#이름과 가격이 순번이 같음

```
#쥬스바의 가격은 마지막 값
#가격에 접근하기 위해 cone["가격"]으로 접근
#그 값중에서 인덱스 3번 조사
```

```
cone["가격"][3]=1200
print(cone,"\\n")
```

```
ice = PD.DataFrame(cone)
print(ice.to_markdown())
```

```
{'이름': ['월드콘', '글레도르', '메로나', '쥬스바'], '가격': [1500, 2400, 1200, 1200]}
```

```
|   | 이름   |   가격   |
|---|:-----|:-----:|
| 0 | 월드콘   |   1500   |
| 1 | 글레도르 |   2400   |
| 2 | 메로나   |   1200   |
| 3 | 쥬스바   |   1200   |
```

#문제 3) 글레도르가 품질되었습니다. 표에서 지워주세요

```
del cone["이름"][1]
del cone["가격"][1]
```

```
print(cone)
```

```
ice = PD.DataFrame(cone)
print(ice.to_markdown())
```

```
{'이름': ['월드콘', '메로나', '쥬스바'], '가격': [1500, 1200, 1200]}
|   | 이름   |   가격   |
|---|:-----|:-----:|
| 0 | 월드콘   |   1500   |
| 1 | 메로나   |   1200   |
| 2 | 쥬스바   |   1200   |
```

▽ 파이썬 6강

▽ 클래스(Class)

현재 프로그래밍에서 하나 또는 시퀀스 형태의 데이터를 저장하고 사용할 수 있음
(변수, 자료구조)

특정 하나의 데이터를 표현하기 위해

데이터가 가지고 있는 속성과 동작을 묶어서 사용하기 위해 클래스를 사용
ex)"학생"에 대한 데이터, "상품"에 대한 데이터

객체(Object)

현실에서 실제로 존재하는 것들

ex)학생, 휴대폰, 자동차...

프로그래밍에서 이러한 객체에 대한 정보를 설계해놓은 것이 클래스이며

이 클래스를 통해 데이터를 만들면 그 데이터는 객체로 불린다

(클래스는 객체를 만들기 위한 틀, 레시피)

객체 지향 프로그래밍(Object - Oriented - Programming : OOP)

프로그램을 설계할 때, 프로그램에서 사용될 핵심 데이터를 '객체'로 만들고 그 '객체'를 중심으로 설계하는 프로그래밍 설계 기법

ex) 라면 끓이는 프로그램 → '라면 자판기' 객체를 만든다 → 자판기에 '라면'의 정보를 넣고 조리를 진행 → '라면' 완성

#클래스 정의하는 방법

```
# class 클래스명 :  
#   변수(클래스 내부에서 만들어지는 변수)  
#   생성자(클래스 생성에 대한 설정 함수)  
#   메소드(클래스 내부에서 만들어지는 함수)
```

#모든 클래스가 해당 요소를 전부 가지진 않는다

#상황에 따라 필요한 부분을 작성하는 것으로 사용 가능

```
class Person:  
    def intro(self):  
        print("안녕하세요")
```

#클래스를 이용해 객체를 생성하는 법

#변수명 = 클래스명()

#클래스명()과 같이 클래스 이름으로 된 함수 형태를 생성자라고 부른다

minsu = Person()

#객체를 통해 클래스의 메소드를 사용
minsu.intro()

1. self는 클래스에서 객체를 의미하는 변수,

클래스 내의 함수를 설계할 경우 self가 첫번째 매개변수로 작성

self는 클래스에서 만든 객체 자신이므로, 따로 값을 호출 위치에서 작성할 필요가 없음

```
class Animal:  
    def cry(self,sound):  
        self.sound = sound  
        print(f"이 동물은 {self.sound}라고 운다")
```

cat = Animal()

dog = Animal()

cat.cry("애옹")

dog.cry("버터!")

이 동물은 애옹라고 운다

이 동물은 버터!라고 운다

#연습 문제) 학생은 이름과 나이를 가지고 있으며, info 메소드를 통해 이름과 나이를 출력할 수 있다

#이름과 나이는 info 메소드를 통해 전달받은 이름과 나이를 가진다

#이 조건에 맞는 클래스를 구현하고 코드를 작성

```
class student:  
    def info(self,name,age):
```

```

self.name = name
self.age = age
print(f"이름은 {self.name}입니다")
print(f"나이는 {self.age}입니다")

```

```

# def name(self, info1):
#     self.info1 = str(info1)
#     print(f"이름은 {self.info1}입니다")
# def age(self, info2):
#     self.info2 = info2
#     print(f"나이는 {self.info2}입니다")

```

```

minsu = student()
minsu.info("민수", 26)

```

```

이름은 민수입니다
나이는 26입니다

```

클래스 내부의 특수 메소드

#__init__(self, 전달값)은 클래스가 생성되고, 값에 대한 초기화 설정에 호출되는 함수(메소드)
 #이 부분을 수정하면 클래스를 생성할 때, 전달해줄 값을 설정할 수 있고
 #그에 대한 초기화를 진행할 수 있음

```

class Product:
    def __init__(self, name, price, id):
        self.name = name
        self.price = price
        self.id = id

```

__str__(self)는 클래스를 통해 만든 객체 그 자체를 문자 형태로 받을 때 return할 값을 설정
 # 따로 수정하지 않았을 경우, 호출 위치와 주소가 나온다

```

def __str__(self):
    return self.name+" "+str(self.price)+" "+str(self.id)

```

#__repr__(self)도 __str__처럼 객체를 문자 형태로 받을 때 사용하는 함수 (Representation)
 #단 str() 프로그램의 사용자를 위해 작성해주는 비 공식적인 문자열 출력에 사용되고
 #repr()는 프로그램의 개발자를 위해 작성해주는 공식적인 문자열 출력에 사용

```

def __repr__(self):
    return f"Product('{self.name}', '{self.price}', '{self.id}')"

```

#__del__(self)는 객체가 소멸했을 때 호출할 메소드

```

def __del__(self):
    print("객체가 폭★8 했습니다")

```

#str과 repr의 비교

#공통점

#두 메소드는 객체의 문자열을 표현하는 기능

#→ 일반적인 문자열은 파이썬을 사용하는

#모든 사람들이 이해할 수 있는 인터페이스 이므로 사람들에게 유익

#차이점

#1. str의 경우는 인자에 대한 문자열화를 진행

ex) a = 1, b = "양념" 이라고 할 때 print(a,b)는 a와 b를 전부 문자열로 변경해서 출력

#이때 a와 b는 각각 str메소드에 의해 내부에서 다른 데이터임에도 문자열로 변환해

#매끄럽게 처리가 되는 방식

#2. repr의 경우는 객체를 평문으로 표현하는 방식

```

# Product1 = Product() #수정을 하면 그 형태에 맞게 값을 넣어줘야 오류가 없음
Product1 = Product("사이다", 1500, 0)

```

```

print(Product1.name)
print(Product1.price)
print(Product1.id)
print(Product1)

```

```
# isinstance(객체, 클래스명)을 통해 해당 객체가 해당 클래스의 결과물(인스턴스)인지 확인
print(isinstance(Product1, Product))
```

```
# ref 출력
print(repr(Product1)) # repr을 통해 객체에 대한 문자열 지정을 하면 repr 호출
print(Product1) # 일반적인 문자열 return 상황일 경우라면 str이 호출

del(Product1)
```

```
사이다
1500
0
사이다 1500 0
True
Product('사이다', '1500', '0')
사이다 1500 0
객체가 폭★8 했습니다
```

```
# 연습 문제) 직사각형을 나타내는 Rectangle 클래스를 다음과 같이 정의하세요
# 처리조건:
# 클래스 변수 : 가로(width), 세로(height)
# 생성자를 통해 가로와 세로에 대한 초기화를 진행

# 클래스 메소드(area): 사각형의 넓이를 계산해주는 기능 (가로*세로)
# 클래스 메소드(circum): 사각형의 둘레를 구하는 기능 (가로+세로)*2
# 객체에 대한 출력 시 클래스명("가로", "세로")로 출력되도록 코드를 수정
```

```
class Rec:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.area = width*height
        self.circum = (width+height)*2
        print(f"Rec(Wn가로:{width}, 세로:{height})")
```

```
R1 = Rec(5,4)
print()
print(R1)
print(str(R1))
print()
print(R1.area)
print(R1.circum)
```

```
Rec(
가로:5, 세로:4)

<__main__.Rec object at 0x79d11d77db50>
<__main__.Rec object at 0x79d11d77db50>

20
18
```

#답변)

```
class Rectangle :
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width*self.height
    def circum(self):
        return (self.width+self.height)*2
    def __repr__(self):
        return f"Rectangle('{self.width}', '{self.height}')"

R2 = Rectangle(5,4)
```

```
print(R2.area())
print(R2.circum())
print(repr(R2))
```

```
20
18
Rectangle('5','4')
```

#연습 문제) 회원 가입 클래스와 회원 정보를 관리하는 클래스를 만들어서
#회원 가입 로그인 기능을 구현합니다

#회원 가입을 진행하면 "회원"이 해당 클래스에 등록되는 방식
#힌트) 자료구조를 활용하세요

```
class Member:
    def __init__(self, id,pw):
        self.id = id
        self.pw = pw

    def __repr__(self):
        return f"Member('{self.id}', '{self.pw}')"

class MemberRegister:

    def __init__(self):
        self.members = [] #리스트를 통해 Member에 대한 추가 기능 구현
        self.current_user = None

    def register(self,member):
        for m in self.members:
            if member.id == m.id:
                print("이미 존재하는 아이디입니다")
                return False
        self.members.append(member)
        print("회원 가입이 되었습니다!")
        return True

    def login(self,id,pw):
        for member in self.members:
            if member.id == id:
                if member.pw == pw:
                    print("로그인 성공")
                    self.current_user = member
                    return True
                else:
                    print("비밀번호 오류")
                    return False
        print("없는 아이디입니다")
        return False

    def logout(self):
        if self.current_user != None:
            print(f"{self.current_user.id} 님 로그아웃 되었습니다.")
            self.current_user = None
            return True
        else:
            print("로그인 상태가 아닙니다.")
            return False

    def memberList(self):
        if self.current_user != None:
            print("Wn--- 현재 회원 명단 ---")
            if not self.members:
                print("등록된 회원이 없습니다.")
            else:
                for member in self.members:
                    print(f"회원명단: {member.id}")
        else :
            print("로그인이 필요합니다")

    def withdraw(self):
        if self.current_user != None:
```

```

print("비밀번호 재입력이 필요합니다")
repassword = input("비밀번호 입력 >> ")
if self.current_user.pw == repassword:

    for i in range(len(self.members)):
        if self.members[i].id == self.current_user.id:
            del self.members[i]
            print(f"{self.current_user.id} 님의 회원 탈퇴가 완료되었습니다.")
            self.current_user = None
            return True

    else:
        print("비밀번호가 일치하지 않습니다")
        return False

    else:
        print("로그인이 필요합니다")
        return False

```

```

system = MemberRegister()

while True:
    print()
    print()
    print("--- 회원 가입 ---")
    print("-- 메뉴 --Wn1. 회원가입Wn2.로그인Wn3.로그아웃Wn4.회원명단Wn5.탈퇴Wn0.종료")
    choice = input("메뉴 선택>> ")
    if choice == '1':
        user_id = input("아이디 입력 >> ")
        user_pw = input("비밀번호 입력 >> ")

        new_member = Member(user_id, user_pw)
        system.register(new_member)
    elif choice == '2':
        user_id = input("아이디 입력 >> ")
        user_pw = input("비밀번호 입력 >> ")
        system.login(user_id,user_pw)
    elif choice == '3':
        system.logout()
    elif choice == "4":
        system.memberList()
    elif choice == "5":
        system.withdraw()
    elif choice == "0":
        print("시스템을 종료합니다")
        break
    else:
        print("다른 번호를 입력하세요")

```

```

--- 회원 가입 ---
-- 메뉴 --
1. 회원가입
2.로그인
3.로그아웃
4.회원명단
5.탈퇴
0.종료
메뉴 선택>> 1
아이디 입력 >> 12
비밀번호 입력 >> 1234
회원 가입이 되었습니다!

```

```

--- 회원 가입 ---
-- 메뉴 --
1. 회원가입
2.로그인

```

```
3.로그아웃
4.회원명단
5.탈퇴
0.종료
메뉴 선택>> 3
로그인 상태가 아닙니다.
```

```
--- 회원 가입 ---
-- 메뉴 --
1. 회원가입
2.로그인
3.로그아웃
4.회원명단
5.탈퇴
0.종료
메뉴 선택>> 2
아이디 입력 >> 12
비밀번호 입력 >> 1234
로그인 성공
```

```
--- 회원 가입 ---
-- 메뉴 --
1. 회원가입
2.로그인
3.로그아웃
4.회원명단
5.탈퇴
0.종료
메뉴 선택>> 3
12 님 로그아웃 되었습니다.
```

```
--- 회원 가입 ---
-- 메뉴 --
1. 회원가입
2.로그인
```

#과제) 완성된 클래스를 기준으로 사용자의 입력을 통해 회원 가입을 진행하는 프로그램 코드를 완성하세요. (다음주

#추가 작업) 로그 아웃 기능을 구현합니다 / 현재 가입된 멤버들의 전체 출력 기능을 구현합니다

#피드백 질문 받을 수 있는 요일 (화,수,주말)

#Ai 버전

```
class Member:
    def __init__(self, id, pw):
        self.id = id
        self.pw = pw # 비밀번호를 그대로 저장 (학습용)

    def __repr__(self):
        return f"Member('{self.id}', '{self.pw}')"

class MemberRegister:

    def __init__(self):
        # 리스트 대신 딕셔너리로 회원 정보 저장: ID(키)와 Member 객체(값)
        self.members_dict = {}
        self.current_user = None

    def register(self, member):

        # 딕셔너리에 해당 ID가 이미 있는지 확인
        if member.id in self.members_dict:
            print("이미 존재하는 아이디입니다")
            return False

        # 새로운 회원을 딕셔너리에 추가
        self.members_dict[member.id] = member
        print("회원 가입이 되었습니다!")
        return True

    def login(self, id, pw):
```

```

# 딕셔너리에서 ID로 회원 객체 가져오기
member = self.members_dict.get(id)

if member: # 해당 ID의 회원이 존재한다면

    if member.pw == pw: # 비밀번호를 직접 비교
        print("로그인 성공")
        self.current_user = member
        return True

    else:
        print("비밀번호 오류")
        return False

print("없는 아이디입니다")
return False

def logout(self):

    if self.current_user != None:
        print(f"{self.current_user.id} 님 로그아웃 되었습니다.")
        self.current_user = None
        return True

    else:
        print("로그인 상태가 아닙니다.")
        return False

def memberList(self):
    if self.current_user != None:
        print("Wn--- 현재 회원 명단 ---")

        if not self.members_dict: # 딕셔너리가 비어있는지 확인
            print("등록된 회원이 없습니다.")

        else:
            # 딕셔너리의 키(ID)들을 순회하며 회원 명단 출력
            for member_id in self.members_dict:
                print(f"회원명단: {member_id}")

    else :
        print("로그인이 필요합니다")

def withdraw(self):
    if self.current_user != None:
        print("비밀번호 재입력이 필요합니다")

        repassword = input("비밀번호 입력 >> ")
        if self.current_user.pw == repassword: # 비밀번호를 직접 비교

            # 딕셔너리에서 현재 로그인된 회원의 ID로 삭제
            # 리스트처럼 인덱스를 찾을 필요 없이, ID를 키로 바로 삭제
            del self.members_dict[self.current_user.id]
            print(f"{self.current_user.id} 님의 회원 탈퇴가 완료되었습니다.")
            self.current_user = None
            return True

        else:
            print("비밀번호가 일치하지 않습니다")

```



```
return False
```

```
else:  
    print("로그인이 필요합니다")  
    return False
```

▽ 파이썬 7강

▽ 파이썬 함수(매개 변수)

함수는 코드 집합체이며, 호출에 의해 사용됨

호출하는 과정에서 요구되는 값을 작성한 것을 매개변수라고 부르며,
사용자는 매개변수에 해당하는 전달해줄 값(인자 값) 을 통해 함수에 값을 전달해줌

```
def greeting(name):  
    print(f"Hello, {name}?")  
  
#매개 변수 = name
```

```
greeting("Killdong")
```

```
#인자 값 = Killdong
```

```
Hello, Killdong?
```

매개변수는 여러개 사용 가능

```
def avg(score1,score2,score3):  
    return(score1+score2+score3)/3
```

```
avg(10,20,30)
```

```
20.0
```

파이썬에서는 매개변수에 값을 지정해주는 디폴트 매개변수를 사용할 수 있음

매개변수 기준으로 뒤쪽으로 몰아서 지정해줘야 하며
값을 지정해주면 그 값에 대한 전달이 없을 때,
지정한 값으로 설정된다.

```
def sum(n = 5):  
    result = 0  
    for i in range(1,n+1):  
        result += i  
    return result
```

```
sum(4)
```

```
10
```

```
sum()
```

```
15
```

#주의사항) 다음과 같은 코드는 불가능함

#1)매개변수 사이에 디폴트 매개변수가 설계되는 경우

```
def sample(a,b=5,c): #불가능  
    pass
```

#2) 앞에서부터 작성하지 않음 (뒤에서 부터)

```
def sample(a=1,b,c): #불가능
    pass
```

```
def sample(a=1,b=2,c): #불가능
    pass
```

#3) 마찬가지로 디폴트 사이에 아닌 값을 넣을 수 없음

```
def sample(a=1,b,c=3): #불가능
    pass
```

#이유) 함수의 매개변수가 여러 개일 경우 인자는 왼쪽부터 순차적으로 넘겨주기 때문

```
def sample(a,b=5,c=3): #가능
    pass
```

▽ 가변 매개변수

함수에 전달할 수 있는 인자의 개수를 조절할 수 있음

전달받은 임의의 개수의 인자는 함수에 tuple(수정불가의 데이터)로써 전달 된다. (positional argument)

전달받은 임의의 개수의 인자는 함수에 dict로써 전달 된다. (keyword argument)

****positional argument(위치 인자)****

```
def 함수명(*가변 매개변수):
    작업할 코드
```

****keyword argument(키워드 인자)****

```
def 함수명(**가변 매개변수):
    작업할 코드
```

```
def add(*args):
    total = 0
    for num in args:
        total += num
    return total
```

```
print(add(1))
print(add(1,2))
print(add(1,2,3))
```

```
1
3
6
```

```
def keywordPrint(**keyword):
    for i in keyword:
        print(f"{i}:{keyword[i]}")
```

```
keywordPrint(name = "홍길동", age = 20, job = "도적")
```

```
name:홍길동
age:20
job:도적
```

#연습문제 1)
#화면에 적힌 실행 예시를 참고하여 함수를 정의합니다.

#조건) 가변 매개변수를 활용할 것

```
#avg(10,11) → 10.5
#avg(1,2,3,4) → 2.5
```

```
def avg(*numbers):
    sum = 0
    for number in numbers:
        sum+=number
    return sum/len(numbers)
```

```
print(avg(10,11))
print(avg(1,2,3,4))
```

```
10.5
2.5
```

▽ 파이썬의 람다식

일반적으로 def로 함수를 따로 정의할 필요가 없거나
짧게 작성하는 것이 더 직관적인 상황에서 사용되는 함수 설계 방법

lambda 매개변수: 표현식

매개변수는 일반 함수처럼 받을 수 있다
표현식의 경우는 하나의 표현식만 가능(여러 줄 작업 불가)
반환 값(return)은 자동으로 처리됨

#1) 기존의 def 함수

```
def add(x,y):
    return x+y
```

```
add(10,20)
```

```
30
```

#2) 람다 바로 사용

```
(lambda x,y:x+y)(10,20)
```

```
30
```

#3) 변수에 저장해서 사용

```
add_lambda = lambda x,y:x+y
```

```
print(add_lambda(10,20))
```

```
30
```

```
def isEven(n):
    if n%2 ==0:
        return "짝수"
    else:
        return "홀수"
```

#def → lambda

```
(lambda n : "짝수" if n%2==0 else "홀수")(10)
```

#파이썬에서 사용되는 람다 예시)

#1) map 함수

#map(함수, 리스트)는 리스트로부터 값 하나하나를 꺼내 함수에 적용시키고 그 결과물을 새로운 리스트에 담아주는 :

```
a = map(lambda x:x**2, range(5))#[0,1,2,3,4]에 x **2를 각각 적용하세요
a =list(a)
print(a)
```

```
list(map(lambda x:x**2, range(5))) #식을 합치는 것도 가능
```

#2) filter 함수

#filter(함수,리스트)는 리스트에 들어있는 값들을 함수에 적용시키고, 결과가 참인 값들만 리스트로 만들어준다
list(filter(lambda x:x<5,range(10))) #[0,1,2,3,4]

#3) 리스트 정렬 (sorted)
numbers = [5,2,6,8,1,9,3,4,10,7]

```
sorted_numbers = sorted(numbers, key = lambda x:x)
print(sorted_numbers)
```

```
[0, 1, 4, 9, 16]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

#문자열 관련 메소드

#>> 가장 빈번하게 많이 사용되는 기능들로 반드시 익혀둘 것

#1) 문자열의 길이

```
sentence = "Hello, My name is Killdong, I'm a famous pop singer"
len(sentence)
```

#사용 예시) 길이 제한
id = input("아이디를 입력해주세요 (최대 6글자)")
if len(id)>6:
 print("아이디는 최대 6글자까지 가능합니다")

#2) 영단어에 대한 대소문자 처리

```
word = "Kimbob"
print(word.upper()) #전체글자 대문자
print(word.lower()) #전체글자 소문자
```

#1) 단어 하나를 대문자로 변경, 그 값을 제외한 나머지 영역을 슬라이싱해서 결합
word = word[0].upper()+word[1:]
print(word)
word = word[0].upper()+word[1:]
print(word)

#2) capitalize() : 문자열의 맨 앞글자 하나를 대문자로 바꾸는 기능

```
word = "kimbob"
word = word.capitalize()
print(word)
```

#공백 제거 strip() : 문자열 양쪽 끝에 있는 공백을 제거하는 기능

```
string = "          hello, world!!          "
string = string.strip()
print(string)
```

#특정 기준으로 문자열을 잘라서 분리하는 기능 split()

```
words = "pen,pineapple,apple"
words = words.split("p")
print(words)
```

#replace(a,b) a를 b로 대체하는 기능

```
string = "za World!!"
new_string = string.replace("World", "Warudo")
print(new_string)
```

#검색 기능

```
word = "Victory"

print(word.startswith("V")) #True 해당 값이 V로 시작하는가?
print(word.endswith("k")) #False 해당 값이 V로 시작하는가?
print(word.find("to")) # "to"에 대한 위치
print(word.index("c")) # "c"의 인덱스 위치
print(word.count("i")) # i의 개수
```

#문자열 판별

```
print("abc123".isalnum()) # True 해당 값이 알파벳+숫자의 조합인가?
print("abc".isalpha()) # True 해당 값이 알파벳인가?
print("123".isdigit()) # True 해당 값이 숫자인가?
print("   ".isspace()) # True 해당 값이 공백만 존재하는가?
```

```
아이디를 입력해주세요 (최대 6글자)ㄸㄸ
KIMBOB
kimbob
Kimbob
Kimbob
Kimbob
hello, world!!
['', 'en,', 'inea', '', 'le,a', '', 'le']
za Warudo!!
True
False
3
2
1
True
True
True
True
```

```
#multiline = """안녕하세요. 파이썬 세계로 오신 걸
#환영합니다.
#파이썬은 비단뱀처럼 매력적인 언어입니다."""
```

```
def multiline2(A):
    A = A.split(" ")

def multiline1(A):
    print(A)

def num(A):
    print(f"{len(A)}")
```

```
multiline1("안녕하세요. 파이썬 세계로 오신 걸\n환영합니다.\n파이썬은 비단뱀처럼 매력적인 언어입니다.")
multiline2("안녕하세요. 파이썬 세계로 오신 걸\n환영합니다.\n파이썬은 비단뱀처럼 매력적인 언어입니다.")
num("안녕하세요. 파이썬 세계로 오신 걸\n환영합니다.\n파이썬은 비단뱀처럼 매력적인 언어입니다.")
```

```
안녕하세요. 파이썬 세계로 오신 걸
환영합니다.
파이썬은 비단뱀처럼 매력적인 언어입니다.
49
```

```
#해답 1)
multiline = "안녕하세요. 파이썬 세계로 오신 걸 환영합니다. 파이썬은 비단뱀처럼 매력적인 언어입니다."

for A in multiline.split():
    print(A)
```

```
print(f"단어 개수: {len(multiline.split())}")
```

```
안녕하세요.  
파이썬  
세계로  
오신  
걸  
환영합니다.  
파이썬은  
비단뱀처럼  
매력적인  
언어입니다.  
단어 개수: 10
```

```
# 해답 2)  
# 목찌빠 게임의 규칙  
# 가위바위보를 진행한다  
# 승자의 턴으로 가며 가위바위보를 한번 더 진행한다  
# 결과가 무승부인 경우 해당 턴 플레이어의 승리  
# 패배인 경우 상대방에게 턴이 넘어간다  
# 승리인 경우 다시 진행한다  
import random  
  
score = False  
i = 1  
  
while score is False:  
    if i == 1:  
        print("간다!!!! 승부다!!!!")  
        print("가위! 바위!! ....!!Wn")  
        print("1.가위 2.바위 3.보")  
        you = input("당신의 선택은..>>WnWn")  
        enemy = random.randint(1,3)  
  
        if you in ["1","2","3"]:  
            you = int(you)  
            if you == enemy:  
                print("무승부다아앗!!")  
                continue  
  
            elif you ==1 and enemy == 3:  
                print("선공은 당신이다!")  
                i = 2  
                continue  
  
            elif you ==2 and enemy ==1:  
                print("선공은 당신이다!")  
                continue  
  
            elif you ==3 and enemy == 2:  
                print("선공은 당신이다!")  
                i = 2  
                continue  
  
            else:  
                print("훗!! 내차례 다앗!!!!")  
                i = 3  
                continue  
        else:  
            print("도망갈 곳은 없다아앗!! 1,2,3 번만 누르란 말이얏!!")  
            continue  
  
    if i == 2:  
        print("다시 승부다!!!!")  
        print("목! 찌!! ....!!Wn")  
        print("1.가위 2.바위 3.보")  
        you = input("당신의 선택은..>>WnWn")  
        enemy = random.randint(1,3)
```

```

if you in ["1","2","3"]:
    you = int(you)

    if you == enemy:
        print("당신의 승리다아앗!!!!")
        score = True
        break

    elif you ==1 and enemy == 3:
        print("아직 당신의 차례다!!")
        continue

    elif you ==2 and enemy ==1:
        print("아직 당신의 차례다!!")
        continue

    elif you ==3 and enemy ==2:
        print("아직 당신의 차례다!!")
        continue

    else:
        print("이번엔 내 차례다아앗!!!")
        i = 3
        continue

if i == 3:
    print("다시 승부다!!!!")
    print("묵! 짜!! ....!!Wn")
    print("1.가위 2.바위 3.보")
    you = input("당신의 선택은..>>WnWn")
    enemy = random.randint(1,3)

    if you in ["1","2","3"]:
        you = int(you)
        if you == enemy:
            print("크흑!!! 패배라니!!!")
            score = True
            break

        elif you ==1 and enemy == 2:
            print("아직 내 차례다!!")
            continue

        elif you ==2 and enemy == 3:
            print("아직 내 차례다!!")
            continue

        elif you ==3 and enemy ==1:
            print("아직 내 차례다!!")
            continue

        else:
            print("훗!! 이번엔 당신의 차례다!!!")
            i = 2
            continue

    else:
        print("도망갈 곳은 없다아앗!! 1,2,3 번만 누르란 말이얏!!")
        continue

```

간다!!!! 승부다!!!!
 가위! 바위!!!!

1.가위 2.바위 3.보
당신의 선택은..>>

1
무승부다아앗!!
간다!!!! 승부다!!!!
가위! 바위!!!!

1.가위 2.바위 3.보
당신의 선택은..>>

1
선공은 당신이다!
다시 승부다!!!!
목! 찌!!!!

1.가위 2.바위 3.보
당신의 선택은..>>

2
아직 당신의 차례다!!
다시 승부다!!!!
목! 찌!!!!

1.가위 2.바위 3.보
당신의 선택은..>>

.
다시 승부다!!!!
목! 찌!!!!

1.가위 2.바위 3.보
당신의 선택은..>>

3
아직 당신의 차례다!!
다시 승부다!!!!
목! 찌!!!!

1.가위 2.바위 3.보
당신의 선택은..>>

1
이번엔 내 차례다앗!!
다시 승부다!!!!
목! 찌!!!!

1.가위 2.바위 3.보
당신의 선택은..>>

2
아직 내 차례다!!
다시 승부다!!!!
목! 찌!!!!

```
import random
```

```
def G(a): # get_hand_name: 숫자를 가위/바위/보 이름으로 변환
    if a == 1: return "가위"
    elif a == 2: return "바위"
    else: return "보"
```

```
def W(a, b): # determine_winner: 승패 판단 (0:무, 1:A승, 2:B승)
    return (a - b + 3) % 3
```

```
S = False # score: 게임 종료 여부
```

```
T = 0      # current_turn_player: 0-가위바위보, 1-사용자 턴, 2-컴퓨터 턴
```

```
print("간다 키시베 로한!!!! 승부다!!!!")
```

```
while True:
    print("\n가위! 바위!! .....!!")
    print("1.가위 2.바위 3.보")
```

```
    while True:
        U = input("당신의 선택은..>> ") # user_input
        if U in ["1", "2", "3"]:
            A = int(U) # user_hand
            break
    else:
```



```

        print("잘못된 입력입니다. 1, 2, 3 중 하나를 입력해주세요. 다시 시도합니다.")

B = random.randint(1, 3) # computer_hand

print(f"당신: {G(A)}, 컴퓨터: {G(B)}")

R = W(A, B) # result

if T == 0: # 가위바위보 턴
    if R == 0:
        print("무승부다아앗!! 다시 가위바위보!")
    elif R == 1:
        print("선공은 내 차지다!")
        T = 1 # 사용자 선공
    else: # R == 2 (사용자 패배)
        print("훗!! 내 차례군 키시베 로한!!!!")
        T = 2 # 컴퓨터 선공

else: # 목찌빠 턴
    if R == 0: # 목찌빠 무승부: 현재 공격자 승리
        if T == 1:
            print("나의 승리다아앗!!!!") # 사용자 승리
        else: # T == 2
            print("크흑!!! 내 패배라니!!!!") # 컴퓨터 승리 (사용자 시점)
            break # 게임 종료

    elif R == 1: # 사용자 승리 (공격권 유지)
        print("아직 내 차례다!!")
        T = 1 # 공격권 사용자에게
    else: # R == 2 (사용자 패배, 공격권 상실)
        print("훗!! 이번엔 내 차례다!!!")
        T = 2 # 공격권 컴퓨터에게

print("\n--- 게임 종료 ---")

```

```

ist = [10,1,5,2]
print(ist)

```

```
[10, 1, 5, 2]
```

```

ist = ist * 2
print(ist)

ist.append(ist[0]*2)
print(ist)

```

```
[10, 1, 5, 2, 10, 1, 5, 2]
[10, 1, 5, 2, 10, 1, 5, 2, 20]
```

```

ist = ist[1::2] #0번, 1번 에서 2칸씩
print(ist)

```

```
[1, 2, 1, 2]
```

```

coin = 58400

Black_coin = coin//10000
Red_coin = (coin%10000)//5000
Green_coin = ((coin%10000)%5000)//1000
Blue_coin = (((coin%10000)%5000)%1000)//100

print(f"현재 소지금은 {coin}원입니다. 해당 소지금을 전부 코인으로 교환하겠습니다.")
print("코인이 교환되었습니다")
print(f"블랙코인 {Black_coin}개 레드코인 {Red_coin}개 그린코인 {Green_coin}개 블루코인{Blue_coin}개")

```

```

현재 소지금은 58400원입니다. 해당 소지금을 전부 코인으로 교환하겠습니다.
코인이 교환되었습니다
블랙코인 5개 레드코인 1개 그린코인 3개 블루코인4개

```

▽ 파이썬 8강

▽ 클래스 상속

기능을 물려주는 용도의 클래스와 물려받아 새롭게 만드는 클래스를 설정

객체에 대한 계층적인 구조를 설계할 때 사용하는 방식

물려주는 쪽(기존 클래스): super class, base class, parent class 물려받는 쪽(상속 클래스): sub class, derived class, child class

```
class 클래스명:
```

```
    ~내용~
```

```
class 자식클래스명(부모클래스명):
```

```
    ~내용~
```

```
class Vehicle :
    def __init__(self, name):
        self.name = name
    def move(self):
        print(f"{self.name}로 이동합니다")
    def stop(self):
        print(f"{self.name}가 정지합니다")
```

```
class Car(Vehicle):
    pass
```

```
class Boat(Vehicle):
    def booster (self):
        print("부스터 가동!!")
    def dropAnchor(self):
        print("닻을 내려라!")
```

```
C = Car("잔용차")
C.move()
C.stop()
```

```
잔용차로 이동합니다
잔용차가 정지합니다
```

상속으로 연결된 클래스는 기존 클래스의 변수와 함수를 그대로 사용할 수 있음

```
# B = Boat():
#     pass
```

기존 클래스의 생성 기능(**init**)가 수정되어 있다면

물려받은 클래스도 해당 생성 기능에서 전달할 값을 전달해야 함

```
B = Boat("배")
B.move()
B.booster()
B.stop()
B.dropAnchor()
```

```
배로 이동합니다
부스터 가동!!
배가 정지합니다
닻을 내려라!
```

✓ 클래스의 다중 상속

Java에서는 클래스의 다중 상속을 허용하지 않음(다이아몬드 상속 문제)
그러나 파이썬에서는 클래스의 다중 상속을 허용한다.

```
#다이아몬드 상속이란..?

class A:
    def check(self):
        print("A")

class B(A):
    def check(self):
        print("B")

class C(A):
    def check(self):
        print("C")

class D(B,C):
    pass

d = D()
d.check()
#D는 B의 check, C의 check를 물려받는다 이 경우 D는 B와 C중 누구의 Check를 물려받을까?
#이 상황에서 A가 2번 호출될 가능성 또는 메소드에 대한 충돌이 발생할 가능성이 있다
#다른 언어에서는 이런 모호성으로 다중 상속을 지원하지 않는 경우도 존재한다(Java)
```

B

✓ 파이썬의 MRO(Method Resolution Order)

상속 계층에서 메소드나 속성을 검색하는 순서를 의미

C3 선형화 알고리즘

다중 상속 시 메소드의 탐색 순서(MRO)를 안정적이고 예측 가능하게 만드는 알고리즘
해당 알고리즘으로 인해 파이썬에서는 다중 상속을 써도 안전하게 사용이 가능

해당 알고리즘의 원리

1. 자기 자신이 먼저 오게 한다
2. 자식 클래스는 부모 클래스보다 먼저 오게 한다
3. 왼쪽에서 오른쪽 순서를 유지한다

장점

1. 중복 제거(동일한 부모를 상속 받을 경우 한번만 호출)
2. 순서 보장(정해진 순서대로 메소드 호출)
3. 다중 상속 충돌 상황 발생 X
4. 순서 개념으로 인해 작동 방식이 예측 가능한 형태로 설계 됨

```
print(D.__mro__)
```

```
(<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>)
```

```
class A:pass
class B(A):pass
class C(A):pass
class D(B,C):pass
```

```
print("A.__mro__:",A.__mro__)
print("B.__mro__:",B.__mro__)
```

```
print("C.__mro__:",C.__mro__)
print("D.__mro__:",D.__mro__)
```

```
A.__mro__: (<class '__main__.A'>, <class 'object'>)
B.__mro__: (<class '__main__.B'>, <class '__main__.A'>, <class 'object'>)
C.__mro__: (<class '__main__.C'>, <class '__main__.A'>, <class 'object'>)
D.__mro__: (<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>)
```

#함수를 통한 출력 시뮬레이터 코드 예시

```
def merge(sequences):
    result = [] #MRO 리스트
    print("start merge...")

    while True:
        #리스트 컴프리헨션
        #[변수 for문 if문]으로 해당 조건에 맞는 값들을 가진 리스트가 구축됨
        sequences = [sequence for sequence in sequences if sequence]
        #빈 리스트에 대한 제거

        if not sequences:
            print("All sequences merge complete")
            return result

        for sequence in sequences:
            candidate = sequence[0] #각 시퀀스의 첫번째 값을 후보로 등록
            tails = [s[1:] for s in sequences if s!=sequence]
            #첫 원소를 제외한 나머지 부분은 후보에서 제외

            if all(candidate not in tail for tail in tails):
                #all(iterable) : 반복이 가능한 객체에 모든 요소가 참인지를 확인하는 기능
                break

        else:
            #후보가 없으면 충돌이 발생했다는 뜻(예외 상황)
            #예외가 발생했음을 알리는 기능 raise Exception("메시지")
            raise Exception("C3 선행화 실패: (순환 또는 다이아몬드 상속 문제 발생)")

        print(f"selected candidate : {candidate}") #선택된 값에 대한 명시
        result.append(candidate) #선택된 값을 결과 리스트에 등록

        #후보로 선택된 값들을 모든 시퀀스에서 제거
        for sequence in sequences:
            if sequence and sequence[0] == candidate:
                sequence.pop(0)

        print(f"result: {result}")
        print(f"sequences: {sequences}")
    return result

#해당 코드는 병합이 가능한 후보들을 찾아서 결과에 추가하는 작업을 진행
```

```
def linearize(cls,parents):
    print(f"{cls} 클래스 MRO")
    sequences = []

    #재귀 코드(함수에서 자기 자신을 다시 호출하는 형태의 설계)
    #parent가 튜플이면 linearize()를 호출해 mro를 다시 계산
    #이미 존재하는 값일 경우 __mro__를 다시 사용
    for parent in parents:
        mro = linearize(parent[0],parent[1]) if isinstance(parent,tuple) else parent.__mro
        sequences.append(list(mro))
    sequences.append(list(parents))
```

```

print(f"{cls} merge sequences")
for sequence in sequences:
    print(sequence)
return [cls] + merge(sequences)

```

```

simulation = linearize("D", [('B', [("A", [])]), ("C", [("A", [])])])
print("최종 MRO: ", simulation)

```

```

#A는 아무값도 상속하지 않는다 → "A", []
#B는 A를 상속한다 → "B", [("A", [])]
#C는 A를 상속한다 → "C", [("A", [])]
#D는 B와 C를 상속한다 → "D", [('B', ["A", []]), ("C", ["A", []])]

```

```

D 클래스 MRO
B 클래스 MRO
A 클래스 MRO
A merge sequences
[]
start merge...
All sequences merge complete
B merge sequences
['A']
[('A', [])]
start merge...
selected candidate : A
result: ['A']
sequences: [], [('A', [])]
selected candidate : ('A', [])
result: ['A', ('A', [])]
sequences: []
All sequences merge complete
C 클래스 MRO
A 클래스 MRO
A merge sequences
[]
start merge...
All sequences merge complete
C merge sequences
['A']
[('A', [])]
start merge...
selected candidate : A
result: ['A']
sequences: [], [('A', [])]
selected candidate : ('A', [])
result: ['A', ('A', [])]
sequences: []
All sequences merge complete
D merge sequences
['B', 'A', ('A', [])]
['C', 'A', ('A', [])]
[('B', [('A', [])]), ('C', [('A', [])])]
start merge...
selected candidate : B
result: ['B']
sequences: [['A', ('A', [])], ['C', 'A', ('A', [])], [('B', [('A', [])]), ('C', [('A', [])])]]
selected candidate : C
result: ['B', 'C']
sequences: [['A', ('A', [])], ['A', ('A', [])], [('B', [('A', [])]), ('C', [('A', [])])]]
selected candidate : A
result: ['B', 'C', 'A']
sequences: [[('A', [])], [('A', [])], [('B', [('A', [])]), ('C', [('A', [])])]]
selected candidate : ('A', [])
result: ['B', 'C', 'A', ('A', [])]
sequences: [], [], [('B', [('A', [])]), ('C', [('A', [])])]
selected candidate : ('B', [('A', [])])
result: ['B', 'C', 'A', ('A', []), ('B', [('A', [])])]
sequences: [[('C', [('A', [])])]]
selected candidate : ('C', [('A', [])])
result: ['B', 'C', 'A', ('A', []), ('B', [('A', [])]), ('C', [('A', [])])]
sequences: []

```

✎ 파이썬 9강

✎ 예외(Exception)

프로그램 실행 중에 발생하는 오류 상황을 의미하며 예외가 발생하면 보통 프로그램이 중단되기 때문에 이를 적절히 처리하는 작업이 필요함

```
try:
    예외가 발생할 가능성이 있는 코드 작성 영역
except 예외_타입 :
    예외가 발생했을 때 실행할 코드 작성 영역
else:
    예외가 발생하지 않았을 경우 실행할 영역
finally:
    항상 실행되는 코드
```

try - except는 항상 같이 있으며, else나 finally는 선택가능

#1) try없이 예외가 발생할 상황의 코드 작성

```
try:
    result = 10/0

except ZeroDivisionError:
    print("0으로 나눌 수 없습니다.")
#오류가 발생 할 수 있는 위치에 try - except 작성
```

```
0으로 나눌 수 없습니다.
프로그램 종료
```

#예시) 데이터 테이블에서 이름으로 사용자를 검색하는 코드 설계
#전제) 실제 데이터 테이블이 아닌 파이썬에서 임의로 만든 데이터베이스 사용(dict)

```
user_database = {
    "Melice" : {"age":25,"email":"melice@sample.com"},
    "Abby" : {"age":26,"email":"Abbdul@sample.com"},
    "Jotaro" : {"age":35,"email":"ora@sample.com"}
}

#검색 기능(함수)

def search_user(name):
    if name not in user_database:
        #raise 에러명(메시지명) : 해당 에러를 강제로 발생시키고, 메시지를 전달
        raise KeyError(f"[ERROR] {name}에 해당하는 사용자를 찾을 수 없습니다.")
    return user_database[name] #일반적인 경우라면 이름 그대로 return

try:
    result = search_user("Jotaro")
    print(f"[SUCCESS] 사용자의 검색 결과는 다음과 같습니다.{result}")
    result = search_user("jojo")
    print(f"[SUCCESS] 사용자의 검색 결과는 다음과 같습니다.{result}")
except KeyError as e: # ???:"KeyError를 e라고 쓰겠어"
    print(e)
```

```
[SUCCESS] 사용자의 검색 결과는 다음과 같습니다.{'age': 35, 'email': 'ora@sample.com'}
'[ERROR] jojo에 해당하는 사용자를 찾을 수 없습니다.'
```

#사용자 커스텀 예외
#파이썬에서 제공해주는 예외와 다른 사용자가 만든 예외를 통해 가독성을 높이고
#프로그램에서 특정 "예외"를 정의해 사용할 수 있어 편리함

#만드는 방법 클래스명(Exception), 즉 Exception에 대한 상속을 진행하는 것을 의미

```
class UserNotFoundError(Exception):
    def __init__(self,username):
        super().__init__(f"[ERROR] {username}에 해당하는 사용자는 없습니다.")
        self.username = username
```

#위에서의 검색 기능만 재수정
#검색기능(함수)

```

def search_users(name):
    if name not in user_database:
        raise UserNotFoundError(name)
    return user_database[name]

try:
    result = search_users("Jotaro")
    print(f"[SUCCESS] 사용자의 검색 결과는 다음과 같습니다.{result}")
except UserNotFoundError as qw: # ???:"KeyError를 e라고 쓰겠어"
    print(qw)

print()
print()

try:
    result = search_users("Qutaro")
    print(f"[SUCCESS] 사용자의 검색 결과는 다음과 같습니다.{result}")
except UserNotFoundError as qw: # ???:"KeyError를 e라고 쓰겠어"
    print(qw)

```

[SUCCESS] 사용자의 검색 결과는 다음과 같습니다.{ 'age': 35, 'email': 'ora@sample.com' }

[ERROR] Qutaro에 해당하는 사용자는 없습니다.

#다음 시나리오를 통해 클래스를 완성하세요

#회원 가입을 진행하는 프로그램 설계

#예외 상황 : 사용자의 이름이 이미 존재할 경우 AlreadyExistError 발생

이메일 안에 @가 없을 경우 EmailFormatError 발생

#아이디, 비밀번호, 이메일 순으로 저장된 데이터베이스

```

user_database = [
    ("Melice", "1234", "melice@sample.com"),
    ("Abby", "1234", "Abbdul@sample.com"),
    ("Jotaro", "1234", "ora@sample.com")
]

test_database = [
    ("Melice", "1234", "melice@sample.com"), #회원 가입 성공
    ("Melice", "1234", "Abbdul@sample.com"), #중복 이름 사용
    ("Jotaro", "oraora", "orasample.com") #@가 없는 경우
]

```

```

class userinfo():
    def __init__(self, ID, PASSWORD, EMAIL):
        self.ID = ID
        self.PASSWORD = PASSWORD
        self.EMAIL = EMAIL

class AlreadyExistError(Exception):
    def __init__(self, ID):
        self.ID = ID
        super().__init__(f"[ERROR] {ID}는 이미 존재하는 사용자입니다.")

class EmailFormatError(Exception):
    def __init__(self, EMAIL):
        self.EMAIL = EMAIL
        super().__init__(f"[ERROR] {EMAIL}은 이메일 형식이 아닙니다.")

```

```

users = [userinfo(ID, PASSWORD, EMAIL) for ID, PASSWORD, EMAIL in user_database]

try:
    result = [userinfo(ID, PASSWORD, EMAIL) for ID, PASSWORD, EMAIL in test_database]
    if ID in user_database:
        raise AlreadyExistError(ID)

except AlreadyExistError as Al:
    print(Al)

try:
    result = [userinfo(ID, PASSWORD, EMAIL) for ID, PASSWORD, EMAIL in user_database]
    if "@" not in EMAIL:
        raise EmailFormatError(EMAIL)
except EmailFormatError as Em:
    print(Em)

```

[ERROR] orasample.com은 이메일 형식이 아닙니다.

#ANSWER)

#1) 예외 클래스 만들기

```

class AlreadyExistError(Exception):
    def __init__(self, ID):
        self.ID = ID
        super().__init__(f"[ERROR] {ID}는 이미 존재하는 사용자입니다.")

class EmailFormatError(Exception):
    def __init__(self, EMAIL):
        self.EMAIL = EMAIL
        super().__init__(f"[ERROR] {EMAIL}은 이메일 형식이 아닙니다.")

```

#2) 회원 가입 함수 만들기

```

def register(ID, PASSWORD, EMAIL):
    #중복된 예외 상황
    for user in user_database:
        if user[0] == ID:
            raise AlreadyExistError(ID)

    #이메일 양식에 대한 예외
    for user in user_database:
        if "@" not in EMAIL:
            raise EmailFormatError(EMAIL)

    #가입 기능(정상적인 경우)
    user_database.append((ID, PASSWORD, EMAIL))
    print(f"{ID}님이 가입하셨습니다")
    print("현재 회원 명단 : ", user_database) #확인용

```

```

for i, (ID, PASSWORD, EMAIL) in enumerate(test_database, start=1):
    try:
        print(f"{i}번째 접속 테스트 = 이름 : {ID} 이메일 : {EMAIL}")
        register(ID, PASSWORD, EMAIL)
    except Exception as e:
        print(e)

```



```
for i in range(len(test_database)):
    try:
        ID,PASSWORD,EMAIL = test_database[i]
        print(f"{i+1}번째 접속 테스트 = 이름 :{ID} 이메일 : {EMAIL}")
    except Exception as e:
        print(e)
```

```
1번째 접속 테스트 = 이름 :Melice 이메일 : melice@sample.com
[ERROR] Melice는 이미 존재하는 사용자입니다.
2번째 접속 테스트 = 이름 :Melice 이메일 : Abbdul@sample.com
[ERROR] Melice는 이미 존재하는 사용자입니다.
3번째 접속 테스트 = 이름 :Jotaro 이메일 : orasample.com
[ERROR] Jotaro는 이미 존재하는 사용자입니다.
1번째 접속 테스트 = 이름 :Melice 이메일 : melice@sample.com
2번째 접속 테스트 = 이름 :Melice 이메일 : Abbdul@sample.com
3번째 접속 테스트 = 이름 :Jotaro 이메일 : orasample.com
```

▼ 컴프리헨션(Comprehension)

직관적으로 리스트를 생성할 때 사용되는 파이썬의 기능

```
#일반적인 배열이나 리스트의 활용
#리스트를 쉽게 짧게 한 줄로 만들 수 있다

#[(변수를 활용할 값) for (변수) in (순회할 수 있는 값)]

size = 10

arr = [i for i in range(10)]

#arr에는 i가 들어가 있고, 그 i는 각각 for문에 의해 작업된 i

print(arr)

arr = [i*i for i in range(1,11)]

print(arr)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

#2) 리스트 컴프리헨션은 내부에서 if 키워드를 지원

```
arr = [x for x in range(1,101) if x%2== 0]
print(arr)
```

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62, 64, 66, 68, 70, 72,
```

#3) 맨 앞에 사용하는 값이 반드시 for문 안에 있는 변수와 같을 필요는 없다
#앞의 표현 값에 대해서도 if문 설계가 가능

```
arr = ["Even" if i%2 == 0 else "Odd" for i in range(1,11)]
print(arr)
```

```
['Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even']
```

#4) for문 뒤에 if문이 여러개인 경우 (and 처럼 작용됨)
arr = [x for x in range(100) if x % 2 ==0 if x % 3 == 0]
print(arr)

```
[0, 6, 12, 18, 24, 30, 36, 42, 48, 54, 60, 66, 72, 78, 84, 90, 96]
```

#5) 리스트 컴프리헨션 or 사용
#if를 한번만 사용하고 or를 따로 기입

```
arr = [x for x in range(10) if x%2==0 or x%3==0]
print(arr)
```

```
# if는 한번만 명시, or만 내부에서 사용
# arr = [x for x in range(10) if x%2==0 or if x%3==0]
```

```
[0, 2, 3, 4, 6, 8, 9]
```

#6) 함수의 return 값으로 조건 분리

```
def div2or3(x):
    return x%2==0 or x%3==0

arr = [x for x in range(10) if div2or3(x)]
print(arr)
```

```
[0, 2, 3, 4, 6, 8, 9]
```

#7) 중첩 for문 설계 가능

#[(변수를 활용할 값) for (변수1) in (순회할 수 있는 값1) for (변수2) in (순회할 수 있는 값2)] 기본 형태
#[(변수를 활용할 값) for (변수1) in (순회할 수 있는 값1) if문 for (변수2) in (순회할 수 있는 값2) if문] 원형

```
result = [(x,y) for x in range (3) for y in range(3)]
print(result)
```

```
soup = ("돈코츠", "시오", "쇼유")
toping = ("차슈", "멘마", '나루토마키')
```

```
ordered_menu = [(s,t) for s in soup for t in toping]
```

```
for menu in ordered_menu:
    print(menu)
```

```
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]
('돈코츠', '차슈')
('돈코츠', '멘마')
('돈코츠', '나루토마키')
('시오', '차슈')
('시오', '멘마')
('시오', '나루토마키')
('쇼유', '차슈')
('쇼유', '멘마')
('쇼유', '나루토마키')
```

딕셔너리 컴프리헨션
dict 형태의 데이터를 대상으로 컴프리헨션 가능

{키 : 값 for 변수 in 순회 가능한 값}

```
keys = ["이름", "나이", "지역"]
values = ["김덕배", 35, "서울"]

person = {k:v for k,v in zip(keys,values)}
print(person)
```

각 수의 제곱에 대한 딕셔너리 생성
squares = {x:x**2 for x in range(1,6)}
print(squares)

```
{'이름': '김덕배', '나이': 35, '지역': '서울'}
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

셋 컴프리헨션
set 형태의 데이터를 대상으로 컴프리헨션 가능

```
numbers = [1,2,2,3,3,3,4,4,4,4]
```

```
cut = {x for x in numbers}
print(cut)
```

```
{1, 2, 3, 4}
```

#연습 문제1) 1부터 20까지의 숫자 중 3 또는 5의 배수인 숫자들의 리스트를 생성하세요

```
number = [x for x in range(1,21) if x%3==0 or x%5==0]
print(number)
```

[3, 5, 6, 9, 10, 12, 15, 18, 20]

#연습 문제2) 리스트 [1,2,3,4,5]에서 각 숫자를 문자열로 변환한 리스트를 만드세요

```
def change(i):
    return f"{i}"
```

```
list = [1,2,3,4,5]
```

```
compre = [x for x in list if change(x) ]
```

```
print(compre)
```

[1, 2, 3, 4, 5]

#연습 문제3) 리스트["apple","banana",'cherry','date']를 각 문자열의 앞글자만 저장하는 리스트를 만드세요
sopung = ["apple","banana","cherry","date"]

```
def first_word(x):
    # 입력받은 문자열(x)의 첫 번째 문자를 반환
    return x[0]
```

```
answer = [first_word(word) for word in sopung]
```

```
print(answer)
```

출력: ['a', 'b', 'c', 'd']

```
arr = [x[0] for x in sopung]
print(arr)
```

['a', 'b', 'c', 'd']
['a', 'b', 'c', 'd']

#연습 문제4) 리스트 컴프리헨션을 활용해 숫자 리스트 중에서 5의 배수인지 판단하는 리스트를 만드세요

```
arr = [1,5,10,15,7,8,9]
check = [True if x %5==0 else False for x in arr]
```

```
print(check)
```

[False, True, True, True, False, False, False]

파이썬 알고리즘

알고리즘은 특정 문제를 해결하기 위한
명확하고 구체적인 절차 또는 방법을 의미하며
입력 값을 받아 목표한 출력 값을 얻기 위해
수행해야 할 일련의 계산 단계를 의미한다

설계의 특징

1. 각 단계가 명확해야 하며, 애매함이 없어야 함
2. 수행 단계에서 무한 루프가 발생하지 않도록 설계
3. 가장 효율적인 방법을 선택해야 함(속도, 자원, 효율)
4. 0개 이상의 입력과 1개 이상의 출력값이 존재

알고리즘을 학습하는 방법

기초적인 문법적인 이해가 선행되어야 함

1. 자료형(int, float, str, list, tuple, dict, set)
2. 제어문(조건/반복문, if for while break continue)
3. 함수(return, def, *args, **kwargs)
4. 내장 함수(sum(), max(), min(), zip(), sorted()...)
5. 슬라이싱 / 인덱싱 (a[0], a[::2], a[1:4].)
6. 리스트 컴프리헨션(ex) [x for x in range(11) if x%2 == 0]

✓ 추가로 고려해야 할 내용들

✓ 특정 데이터(자료구조) 와 특정 알고리즘에 대한 이해

자료구조 유형

1. 배열 / 리스트 : 최대 최소값, 구간의 합
2. 해시(dict / set) : 중복 제거, 등장 횟수 체크
3. 스택 / 큐 : 괄호 검사, 데이터 제거 순서(LIFO, FIFO)

자주 사용되는 알고리즘

1. 문자열 관련 : 회문 / 애너그램 / 문자열 압축
2. 재귀 함수 : 팩토리얼 / 피보나치 / 하노이의 탑 / DP
3. 이진 탐색 : 정렬
4. 완전 탐색 : 백트래킹 : 탐색(순열, 조합, 부분 집합...)
5. 거리 계산(그래프) : DFS, BFS (미로탈출 / 영역 개수 세기 / 가장 가까운 지점)
6. 정렬 알고리즘 : 버블, 선택, 삽입, 병합, 퀵...

시간 복잡도와 공간 복잡도

코드의 결과는 처리되나 비효율적인 설계가
속도의 저하와 과도한 메모리 사용으로 이어지기에
더 좋은 값에 대한 판단이 필요하다

빅 O 표기법

최악의 상황을 기준으로 성능을 표현하는 표기법

표기법	이름	설명
$O(1)$	상수	입력 크기와 상관없이 일정한 시간
$O(\log n)$	로그	점점 증가(값이 많을 수록 효율적)
$O(n)$	선형	크기만큼 반복(값이 많을 수록 비효율적)
$O(n \log n)$	로그 선형	대부분의 정렬 알고리즘
$O(n^2)$	이차	이중 for문
$O(2^n)$	지수	입력값이 늘어나면 기하급수적으로 증가

ex) 루프의 개수를 세면 시간 복잡도 자체 판단 가능

1번만 도는 루프 상황 $\rightarrow O(n)$
루프 안에 루프 $O(n^n)$
루프의 횟수가 고정인 경우 $O(1)$
입력 값을 점차 줄여나감 $O(\log n)$ 반복이 절반으로 줄면 $\log n$
 n 번 반복하고 있는데 그 안에서 $\log n$ 이 반복하는 경우 $n \log n$

$\log n$ 인 경우

```
def log_n(n):  
    count = 0  
    while n>1:  
        n=n//2  
        count+= 1
```

```
    return count
print(f"값 1000000에 대한 실행 횟수 : {log_n(1000000)}")
print(f"값 100000에 대한 실행 횟수 : {log_n(100000)}")
print(f"값 10000에 대한 실행 횟수 : {log_n(10000)}")
print(f"값 1000에 대한 실행 횟수 : {log_n(1000)}")
```

```
값 1000000에 대한 실행 횟수 : 19
값 100000에 대한 실행 횟수 : 16
값 10000에 대한 실행 횟수 : 13
값 1000에 대한 실행 횟수 : 9
```